第 7 章　查找

7.1 引言

在日常生活中，人们几乎每天都要做“查找”的工作。例如，在搜索引擎上查找基于某关键字的信息；在手机的通讯录中查找某人的电话号码；在图书馆的数据库中查找某书的库存量或摆放位置；在网购平台上查找自己心仪的商品等等。这些待查找的数据或记录所构成的集合就称为查找表，而所谓查找，就是在一个包含有众多记录的查找表中找出某“特定”记录的过程。此前我们已经学习了各种类型的线性、非线性数据结构，在本章中，我们将讨论适用于不同类型查找表的查找算法，并分析它们的查找效率，从而强调数据结构与算法效率之间的关系。

7.2 查找表的定义与抽象数据类型

查找表（search table）是由同一类型的数据元素（或记录）构成的集合。由于集合中的元素之间存在着非常松散的关系，因此查找表是一种非常灵便的数据结构类型。对查找表可以进行的操作包括：

（1） 查询：确定某个特定数据元素（或记录）是否存在于查找表中。

（2） 检索：查看某个特定数据元素（或记录）的各种属性。

（3） 插入：在查找表中插入一条数据元素（或记录）。

（4） 删除：在查找表中删除一条数据元素（或记录）。

若对查找表只能做前两种操作，此类查找表称为**静态查找表**（static search table）；以上四种操作均可实现的查找表称为**动态查找表**（dynamic search table）。

关键字（key）是数据元素（或记录）中某个数据项，它可以用来标识一个数据元素（或一条记录）。若此关键字可以唯一地标识一条记录，则称之为**主关键字**（primary key）；反之，可以用来标识若干条记录的关键字称为**次关键字**（secondary key）。

查找（searching）就是根据给定数值，在查找表中确定关键字等于该给定数值的数据元素（或记录）是否存在的过程。如果存在这样的记录，则**查找成功**，此时查找结果应为指出记录在查找表中的位置，或者给出记录中包含的所有数据项；若查找表中不存在这样的数据元素，则**查找不成功**，此时的查找结果应该是返回一个“空”记录或者“空”指针。

平均查找长度（Average Search Length，ASL）是指在查找过程中，将查找表中的关键字与给定值进行比较的次数的平均值。因为查找算法的基本操作是将查找表中的数据元素（或记录）的关键字与给定值进行比较，所以平均查找长度的大小可以用来衡量查找算法的性能。对于含有 n 个数据元素（或记录）的查找表，查找成功时的平均查找长度为：

$$ASL = \sum_{i=1}^{n} P_i C_i \qquad (7.1)$$

其中P_i为查找表中第i个记录被查找的概率，且$\sum_{i=1}^{n} P_i = 1$；C_i为在成功找到第i个记录的关键字与给定值相同时，已经与给定值做过比较的关键字的个数。显然，C_i的取值与查找算法相关。

查找表的抽象数据类型，定义如下：

```
ADT SearchTable
{
    数据对象 D：D 是具有相同特性的数据元素的集合，各个数据元素均含有类型相同且可唯一标识数据元
                素的关键字
    数据关系 R：  数据元素同属一个集合
```

基本操作 P：

// **1. 初始化和销毁操作**

Create(&ST, n)

操作结果：构造一个含有 n 个数据元素的查找表

Destroy(&ST)

初始条件：查找表 ST 存在

操作结果：销毁查找表 ST

// **2. 访问型操作**

Search(ST, key)

初始条件：查找表 ST 存在，key 为待查找的关键字

操作结果：若 ST 中包含有关键字等于 key 的记录，则返回该记录的位置；否则返回“空”

Traverse(ST)

初始条件：查找表 ST 存在

操作结果：按某种次序对表中的所有记录访问一次仅且一次

// **3. 加工型操作**

Insert(&ST, e)

初始条件：查找表 ST 存在，e 为待插入的记录

操作结果：若 ST 中不存在关键字等于 e.key 的记录，则插入 e 到 ST 中

Delete(&ST, key)

初始条件：查找表 ST 存在，key 为待删除的记录的关键字

操作结果：若 ST 中存在关键字等于 key 的记录，则删除之

} // SearchTable

7.3 顺序表的查找

静态查找表类似于全国的电话区号、邮政编码等，我们经常在其中进行数据的查找，而表本身不会发生变化。一般的静态查找表采用顺序存储的线性表表示。

基于顺序结构的静态查找表存储结构定义如下：

```
typedef struct
{
    ElemType *SeqList;    // 顺序表存储空间基地址，建表时按需分配存储空间，一般 0 号单元留空
    int length;           // 查找表的长度
} SearchTable;            // 顺序查找表的类型定义
```

7.3.1 顺序查找

顺序查找是一种最简单的查找算法。它的基本过程是：从查找表的一端开始，顺序遍历查找表，依次将遍历到的关键字与给定的数值 key 进行比较。若当前遍历到的关键字与 key 相等，则查找成功；若遍历结束后，仍未找到关键字等于 key 的记录，则查找失败。该顺序查找可以用如下算法实现。

算法 7.1 顺序查找算法

```
int SeqSearch( SearchTable R, KeyType key )
{
    R.SeqList[0].key = key;                          // 0 号单元作为哨兵使用
    for( i=R.length; R.SeqList[i].key!=key; i--) ;   //从后向前做顺序查找
    return i;                                        // 返回查找结果，查找失败则返回为 0
}// SeqSearch
```

这个算法采用从后向前的遍历顺序，并使用查找表中的 0 号单元作为哨兵，可以免去查找过程中每一步都需要判断计数器i是否已经超过查找表的表长。这里 0 号单元起到了一个监视哨的作用。这仅是一个编程技巧上的改进，并不能本质上改变算法的时间复杂度或者查找的ASL。但实践证明，

这个小小的改进可以使得顺序查找在表长超过 1000 时，明显减少了一次查找所需的平均时间，极大地提高了查找的效率。

对于顺序查找的查找性能，若顺序表表长为n，则从其算法过程可以看出，其C_i的大小取决于所查记录在顺序表中的位置。例如，若查找的是表中最后一个元素，那么$C_i = 1$；而如果要查找的是表中第一个元素，则$C_i = n$。一般情况下$C_i = n - i + 1$。假设每个查找表中的每个数据被查找的概率相等，即$P_i = 1/n$，则在等概率情况下，顺序查找的平均查找长度ASL为：

$$ASL = \sum_{i=1}^{n} P_i C_i = \frac{1}{n}\sum_{i=1}^{n}(n - i + 1) = \frac{n+1}{2} \qquad (7.2)$$

这是查找成功的情况，那对于查找不成功的情况呢？如果给定的数值在查找表中并不存在，则查找不成功，此时表中所有的关键字都与给定值进行了比较，包括哨兵。因此查找不成功的C_i均等于$n + 1$，则等概率查找下，其平均查找长度ASL也即等于$n + 1$。这也是顺序查找的最坏情况，因此顺序查找的时间复杂度为$O(n)$。

在实际情况中，有时候顺序查找未必是等概率的。例如，在通讯录中，关系亲密的好友、家人被查找的概率必定会高于关系普通的人；在医院的病例档案中，发病率高、传染性强的病例被查找的概率也会高于那些罕见病的病例。在这种情况下，为了减少比较次数、提高查找效率，应该将表中的记录按照其查找概率逆序排序：查找概率高的放在查找表的末端，反之亦然。但是，在大多数情况下，每个记录被查找的概率无法事先获知。为了解决这个问题，可以在每条记录中再增设一个域，用于保存该条记录被查找的频次，使得顺序查找表始终按照该频次逆序排序。这样，在查找过程中，经常被查找的记录将随着查找频次的不断增加而被不断后移，以便在之后的查找中节省比较的次数。

7.3.2 折半查找

上述顺序查找的时间复杂度为$O(n)$，原因在于表中数据的无序性，即每次都需要遍历整个查找表。如果数据是按照 key 有序排序的，那么是否可以设计一个新的算法策略以降低时间复杂度呢？人们由此提出了折半查找方法。

折半查找又叫作二分查找，要求顺序表按关键字有序（假设是递增有序）。其基本思想是：先确定待查记录在查找表中的区间范围，然后每次折半这个区间以逐步缩小范围，直到找到待查记录或区间用尽为止。

7.3.2.1 折半查找的基本过程

以一个包含 11 条记录的有序查找表（关键字即为数据元素的值）为例，现在要查找关键字分别为 34 和 76 的数据元素。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

要进行折半查找，首先需要设定三个变量low、$high$和mid。low表示查找范围的下界，$high$表示查找范围的上界，mid表示low和$high$所限定的区间的中间位置，即$mid = \lfloor(low + high)/2\rfloor$。那么在初始情况下，$low$和$high$的取值则分别为 1 和 11，表示当前的查找范围是[1,11]，此时mid的取值为 6。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81
↑ low					↑ mid					↑ $high$

key=34 的查找过程如下：

（1） 将查找表中mid位置上的元素的关键字与 key 进行比较。因为 R.SeqList[mid].key>key 且查找表递增有序，所以若 key 存在，则其必然在区间[low，$mid-1$]的范围内。因此更新下一步的查找范围，即令$high=mid-1=5$，$low=1$保持不变，重新计算$mid=\lfloor(1+5)/2\rfloor=3$。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑ low　　↑ mid　　↑ $high$

（2） 仍将查找表中mid位置上的元素的关键字与 key 进行比较。因为 R.SeqList[mid].key<key，所以若 key 存在，则其必然在区间[$mid+1$，$high$]的范围内。因此继续更新下一步的查找范围，即令$low=mid+1=4$，$high=5$保持不变，重新计算$mid=\lfloor(4+5)/2\rfloor=4$。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑↑ low mid　　↑ $high$

（3） 继续将查找表中mid位置上的元素的关键字与 key 进行比较。因为 R.SeqList[mid].key 与 key 相等，所以查找成功。当前的mid即表示待查找数值在查找表中的位置。

可见，每进行一次查找，就舍弃当前区间的一半查找空间，这也是“折半”的含义所在。

key=76 的查找过程如下：

（1） 初始情况与查找 34 时相同，即low和$high$的取值则分别为 1 和 11，表示当前的查找范围是[1,11]，$mid=\lfloor(low+high)/2\rfloor=6$。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑ low　　↑ mid　　↑ $high$

（2） 由于 R.SeqList[mid].key<key，所以令$low=mid+1=7$，$high=11$保持不变，$mid=\lfloor(7+11)/2\rfloor=9$。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑ low　　↑ mid　　↑ $high$

（3） 由于此时 R.SeqList[mid].key<key，所以令$low=mid+1=10$，$high=11$保持不变，$mid=\lfloor(10+11)/2\rfloor=10$。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑↑ low mid　　↑ $high$

（4） 此时 R.SeqList[mid].key 还是小于 key，继续令$low=mid+1=11$，$high=11$保持不变，此时mid也等于 11。

1	2	3	4	5	6	7	8	9	10	11
12	24	30	34	41	48	59	62	70	74	81

↑ low
↑ mid
↑ $high$

（5） 由于 R.SeqList[mid].key>key，于是令$high = mid - 1 = 10$，$low = 11$保持不变。此时作为区间上界的$high$比区间下界low还要小，说明该区间不存在，因此查找表中不存在关键字等于 key 的记录，查找不成功。

根据上述查找成功和查找失败的举例，我们可以看出算法执行的策略。由此可得到折半查找的算法实现如下。

算法 7. 2 折半查找算法

```
int BinarySearch( Stable R, KeyType key )
{    // 折半查找算法
     int low=1, high=R.length, mid;              // 设置初始查找区间
     while( low<=high )                          // 只要查找区间存在
     {
          mid = (low+high)/2;                    // 计算查找区间的中间位置
          if( R.SeqList[mid].key==key )          // 查找成功
               return mid;
          else if( R.SeqList[mid].key<key )      // 继续在后半区查找
               low = mid+1;
          else                                   // 继续在前半区查找
              high = mid-1;
     } // end while
     return 0;                                   // 查找不成功
}// BinarySearch
```

7. 3. 2. 2 折半查找的性能分析

折半查找的查找过程可以用图 7.1 所示的二叉树来描述，该二叉树也称为**判定树**。二叉树的每个结点代表查找表中的一个记录，结点中的数字表示该记录在查找表中的位置。图中的虚线表示查找关键字为 34 和 76 时的查找路径。例如在查找 34 时，由于其位于查找表中的 4 号位置，所以查找过程是一条从根结点到结点 4 的路径。由上述查找过程可知，如果要查找的是 6 号位置上的元素，则只需要做一次比较即可；如果要查找的是位于 3 或者 9 号位置上的元素，则只需要进行两次比较；如果要找位于 1、4、7 或 10 号位置上的元素，则需要进行三次比较；如果要查找位于 2、5、8 或 11 号位置上的元素，则需要进行四次比较。由此可见，查找每个结点所需要的比较次数与该结点在判定树中的层次有关。因此，在查找成功的情况下，折半查找所需要的最多比较次数不会超过树的深度。那么包含有n个结点的判定树的高度等于多少呢？判定树不一定是完全二叉树，但是它的叶子结点所在层次之差最多为 1，且n个结点的判定树的深度与n个结点的完全二叉树的深度是相同的。由此可知，判定树的深度为$\lfloor log_2 n \rfloor + 1$，故而折半查找的时间复杂度为$O(\log n)$。

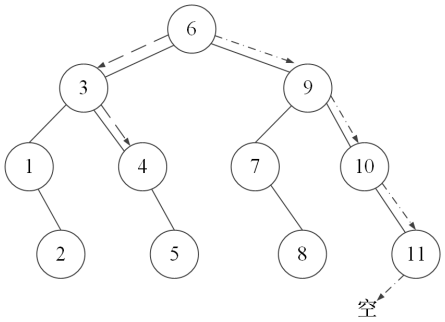


图 7. 1 折半查找过程示意图

以上是折半查找成功的情况，那么查找不成功的情况呢？根据图 7.1 所示的在查找关键字 76 时走过的查找路径可以发现，如果查找路径走向一个不存在的结点，那么查找不成功。若将图 7.1 所

示的判定树中真正存在的结点称为内部结点，而内部结点指向的空域称为外部结点，那么折半查找不成功就意味着走了一条从根结点到外部结点的查找路径。如图 7.2 所示，虚线表示的结点均为外部结点。因此折半查找不成功时，所需的比较次数最多不超过的判定树的深度。

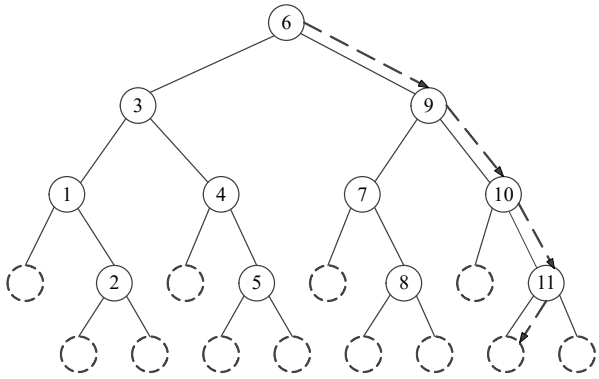


图 7.2 加上外部结点的折半查找过程示意图

那么折半查找的平均查找长度是多少呢？为了讨论方便，假定查找表的长度$n = 2^h - 1$，即其判定树是一个深度等于h的满二叉树。那么在该判定树中，根据二叉树的属性可知，第一层包含 1 个结点，第二层包含 2 个结点，……，第h层包含2^{h-1}个结点。若进行等概率查找，即$P_i = 1/n$，则在查找成功时，折半查找的平均查找长度为

$$ASL = \sum_{i=1}^{n} P_i C_i = \frac{1}{n}\sum_{j=1}^{h}(j \times 2^{j-1}) = \frac{n+1}{n}\log_2(n+1) - 1 \qquad (7.3)$$

当n较大时，可有近似结果：

$$ASL = \log_2(n+1) - 1 \qquad (7.4)$$

可见，折半查找比顺序查找的效率要高。但折半查找只能用于有序的顺序存储结构，对有序的链式存储结构无法进行折半查找，原因请读者自行思考。

7.3.3 索引查找

索引查找又称为分块查找，是顺序查找的一种改进方法。在索引查找中，除了查找表本身以外，还需要额外的空间来创建一个“索引表”。如图 7.3 所示，查找表本身长度为 20，可将该表分为 4 个子表（块），即$(R_0, R_1, \ldots, R_4)$、$(R_5, R_6, \ldots, R_9)$、$(R_{10}, R_{11}, \ldots, R_{14})$和$(R_{15}, R_{16}, \ldots, R_{19})$。为每个子表建立一个索引项，每个索引项包含两个域：一个是关键字，等于该子表内最大的关键字的值；另一个域是指针域，指向该子表的第一条记录在查找表中的位置。索引表按关键字有序，查找表整体则是分块有序。所谓分块有序是指：第二个子表中的所有关键字均大于第一个子表中最大关键字；第三个子表中的所有关键字均大于第二个子表的最大关键字，以此类推。

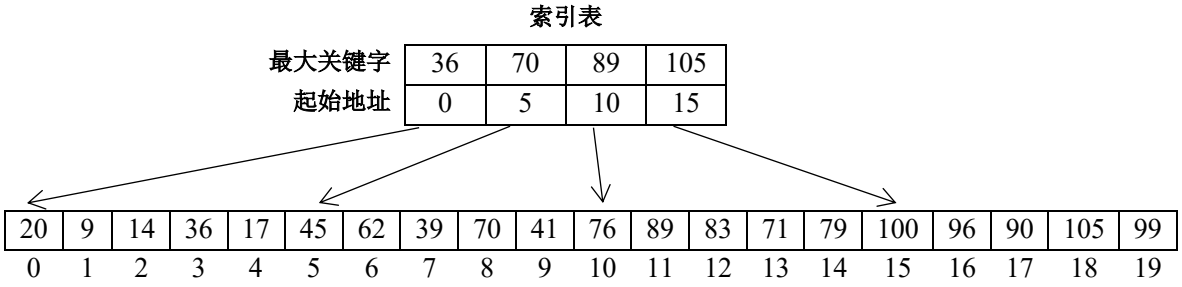


图 7.3 索引查找的存储结构

因此，索引查找需要分两步进行，即首先确定待查关键字 key 所在的子表，然后在相应的子表中进行顺序查找，最终确定 key 的具体位置。例如，若要查找关键字等于给定值 key=71 的记录，可以首先在索引表中顺序查找，因为 70<key<89，所以关键字 71 若存在，则必然在第三个子表

中。由于第三个子表在查找表中的起始位置是 10，则从查找表第 10 号位置开始顺序查找，判断在该子表范围内是否有关键字等于 71，最后返回相应的查找结果即可。

由于索引表是按关键字有序的，因此在确定待查找数值所在子表时，既可以对索引表进行顺序查找，也可进行折半查找；而子表中的数据并非有序，因此在子表中只能进行顺序查找。因此，索引查找算法其实是折半查找与顺序查找的简单组合，其性能介于这两种算法之间。设查找表表长为n，将其平均分为s个子表，每个子表长度即为$b = \lceil n/s \rceil$，如果是等概率查找，则每个子表被查找的概率是$1/s$，子表中每个元素被查找的概率是$1/b$。

若在索引表和子表中均采用顺序查找，则索引查找的平均查找长度为：

$$ASL = \frac{1}{s}\sum_{i=1}^{s} i + \frac{1}{b}\sum_{i=1}^{b} j = \frac{s+1}{2} + \frac{b+1}{2} = \frac{1}{2}\left(s + \frac{n}{s}\right) + 1 \qquad (7.5)$$

可见其平均查找长度不仅与表长n有关系，也与子表的数量s有关系。在给定n的情况下，可以证明，当s取$\sqrt{n}$时，上式的ASL会得到最小值$\sqrt{n}+1$。这个值与顺序查找相比有了很大提升，但还是远不及折半查找。

如果在索引表中做折半查找，在子表中做顺序查找，此时的索引查找其平均查找长度为

$$ASL \approx \log_2(s+1) + \frac{b}{2} \approx \log_2(s+1) + \frac{n}{2s} \qquad (7.6)$$

7.4 树表的查找

以二叉树或者树作为组织结构的查找表称为树表。在对树表进行删除或者插入操作时，可以方便地维护查找表的有序性，而不需要对查找表中的记录进行移动。常见的树表有二叉排序树、平衡的二叉排序树、B-树和 B+树等。

7.4.1 二叉排序树

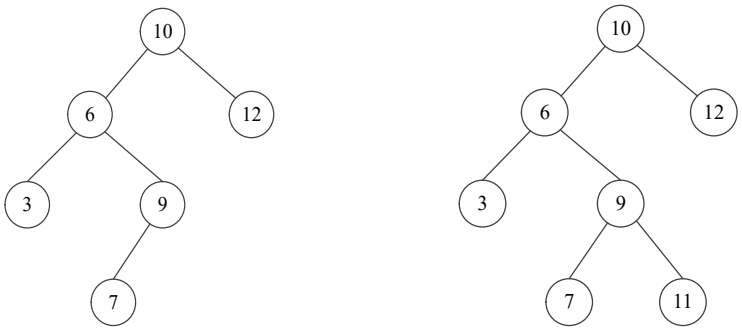
二叉排序树（Binary Sort Tree, BST）是一棵二叉树，满足二叉树的所有属性。除此之外，它还满足以下条件：

（1）若左子树非空，则左子树上所有结点的值均小于根结点的值；

（2）若右子树非空，则右子树上所有结点的值均大于根结点的值；

（3）左、右子树本身也都分别是二叉排序树。

在图 7.4 中，只有图 a 是一个二叉排序树，而图 b 并不是，因为在其左子树中有一个结点的数值为 11，大于根结点的数值 10。从二叉排序树的性质可知其一个重要推论——若按中序遍历的顺序输出二叉排序树中各个结点的数值，则会得到一个递增的有序序列。



a) 二叉排序树　　b) 不是二叉排序树

图 7.4 两棵二叉树

二叉排序树通常采用二叉链的存储结构进行存储，其结点类型定义如下：

```
typedef struct node
```

```
{
    KeyType key;              // 关键字项
    InfoType data;            // 其它数据域
    struct node *left,*right; // 左、右孩子指针
}BSTNode;                     // 二叉排序树结点类型定义
```

7.4.1.1 二叉排序树的查找算法

在二叉排序树上进行查找的过程为：若二叉排序树非空，将待查找关键字 key 与根结点的关键字进行比较。若二者相等，则查找成功；若当前根结点关键字小于 key，则进入根结点的右子树继续做查找；若当前根结点关键字大于 key，则进入根结点的左子树继续查找；若当前根结点为空，则查找不成功。这显然是一个递归的过程。对应的递归算法如下：

算法 7.3 二叉排序树的递归查找算法

```
BSTNode *BSTSearch( BSTNode *T, KeyType key )
{
    if(!T)  return NULL;                    // 查找不成功
    else if( T->key==key )   return T;      // 查找成功
    else if( T->key<key )
        return BSTSearch( T->right,key );   // 进入右子树继续查找
    else
        return BSTSearch( T->left,key );    // 进入左子树继续查找
}// BSTSearch
```

显然，这个查找过程是从根结点出发，在二叉排序树中自顶向下查找，不存在向上的回溯。所以，也可将其方便地改成非递归算法实现。

算法 7.4 二叉排序树的非递归查找算法

```
BSTNode *BSTSearch( BSTNode *T, KeyType key )
{
    BSTNode *p=T;
    while(p)
    {
        if( p->key==key )  return p;      // 查找成功
        else if( p->key<key )
            p = p->right;                 // 进入右子树做查找
        else
            p = p->left;                  // 进入左子树做查找
    }
    return NULL;                          //  查找不成功
}// BSTSearch
```

7.4.1.2 二叉排序树的插入结点算法

当二叉排序树中不存在某个给定值的结点时，可以进行插入操作。新插入的结点一定是叶子结点，而且应该是查找不成功时的查找路径上访问的最后一个结点的左孩子或者右孩子结点。因此，插入是在查找的过程中进行的，可以用递归的方式来描述：若原二叉排序树是空树，则直接插入结点作为根结点；否则如果待插入的关键字小于根结点关键字，则进入其左分支进行插入，相反则进入右分支进行插入。算法实现如下。

算法 7.5 二叉排序树的递归插入算法

```
BSTNode *BSTInsert( BSTNode *T, KeyType key )
```

```
{
    if( T==NULL )                                   // 如果遇到空指针，表明查找不成功，可以进行插入
    {
        T = (BSTNode *)malloc(sizeof(BSTNode));     // 为新结点分配空间
        T->key = key;                               // 为新结点赋值
        T->left = T->right = NULL;                  // 新插入的结点是叶子结点，所以左、右分支域皆为“空”
        return T;                                   // 返回新插入结点的位置
    }
    else if( T->key==key )  return T;               // 如果 key 已经存在，则不进行插入，直接返回
    else if( T->key>key )                           // 如果 key 小于根结点
         T->left = BSTInsert( T->left, key );       // 进入左分支做插入
    else                                            // 如果 key 大于根结点
         T->right = BSTInsert( T->right,key );      // 进入右分支做插入
    return T;
}// BSTInsert
```

如果用非递归的算法来实现，那么至少需要两个指针变量 p 和 pa：用指针 p 在二叉排序树中查找，pa 表示 p 的父结点。如果需要插入一个新结点，则查找一定是不成功的。查找不成功意味着最后指针 p 应该指向“空”，而此时它的父结点 pa 的左孩子或者右孩子就是要插入的新结点的位置。算法如下。

算法 7.6 二叉排序树的非递归插入算法

```
BSTNode *BSTInsert(BSTNode *T,KeyType key)
{
    BSTNode *p=T, *pa=NULL;                     // 定义指针变量并赋值
    // 第 1 步：找到待插入的位置
    while(p)                                    // 如果树非空，则进行查找
    {
        if( p->key==key )  return T;            // 如果 key 已经存在，则不进行插入，算法结束
        else if( p->key<key )                   // 如果当前访问结点小于 key
        {
            pa = p;                             // 更新 pa 指向
            p  = p->right;                      // 继续到右分支进行查找
        }
        else                                    // 如果当前访问结点大于 key
        {
            pa = p;                             // 更新 pa 指向
            p  = p->left;                       // 继续到左分支进行查找
        }
    } // end while
    // 第 2 步：插入新结点
    p = (BSTNode *)malloc(sizeof(BSTNode));     // 为新结点分配空间
    p->key = key;                               // 为新结点赋值
    p->left = p->right = NULL;                  // 新结点是叶子结点
    if( !pa ) return p;                         // 如果原先的树是空树，新插入的结点就是根结点
    if( pa->key<key )                           // 如果原树非空，且 key 大于查找路径上访问的最后一个结点
        pa->right = p;                          // 将新结点插入最后一个结点的右孩子位置
    else                                        // 否则原树非空，且 key 小于查找路径上访问的最后一个结点
        pa->left = p;                           // 将新结点插入最后一个结点的左孩子位置
    return T;
}// BSTInsert
```

7.4.1.3 二叉排序树的构造算法

创建一个二叉排序树，可通过依次输入数据元素并将它们逐一插入到二叉排序树中的合适位置来完成。如果输入 n 个数据元素，则根据输入顺序创建相应的二叉排序树的具体过程如下。

算法 7.7 二叉排序树的创建算法

```
BSTNode *CreateBST(int n)
{
    BSTNode *T=NULL;            //变量 T 表示要创建的二叉排序树的根结点，初始值为空
    keyType key;
    for( i=1; i<=n; i++ )
    {
        scanf(keyType,&key);    // 输入当前要插入的数据元素的关键字
        T = BSTInsert( T, key );  // 调用插入函数将该关键字插入到根结点为 T 的二叉排序树中
    }
    return T;                   // 返回二叉排序树的根结点，创建完成
}// CreateBST
```

7.4.1.4 二叉排序树的结点删除算法

在二叉排序树中删除一个结点 p，若其父结点为 f，则可以分三种情况来讨论：

（1） 结点 p 的度等于零，即 p 是叶子结点。删除叶子结点不会影响二叉排序树的性质，因此可以直接将该叶子结点删除，并修改其父结点 f 对应的指针域指向即可。如图 7.5a 所示，若要删除叶子结点“3”，则由于它是其父结点的左孩子，因此修改其父结点的左分支指向为“空”，然后释放该结点即可完成删除。

（2） 结点 p 的度等于 1，即结点 p 只有左孩子或只有右孩子。如果结点 p 是其父结点的左孩子，那么直接将结点 p 的孩子作为 f 的左孩子，然后删除结点 p 即可；如果 p 是父结点 f 的右孩子，那么将结点 p 的孩子直接作为 f 的右孩子，然后删除结点 p 即可。如图 7.5b 所示，若要删除结点“9”，则由于该结点是其父结点的右孩子，而且该结点只有一个左孩子，因此将其父结点的右分支接指向结点“9”的左孩子，然后释放结点“9”，即可完成删除。

（3） 结点 p 的度等于 2。此时有两种删除方式。一种是在 p 的左分支中寻找一个最大值结点 r，以此最大值覆盖结点 p，然后删除这个结点 r。由于结点 r 必然没有右孩子，因此删除结点 r 的情况与上述的情况 2 相同。另一种方式是在 p 的右分支中寻找一个最小值结点 q，以此最小值覆盖结点 p，然后删除这个结点 q。由于结点 q 必然没有左孩子，因此删除结点 q 的情况与上述的情况 2 相同。如图 7.5c 所示，要删除的是结点“10”，如果采取第一种方式，用其左分支的最大值来做替换，该最大值必然在其左分支的最右端，是结点“9”，即用“9”来替换掉“10”，然后删除结点“9”，该过程与上述删除结点“9”的过程完全相同。

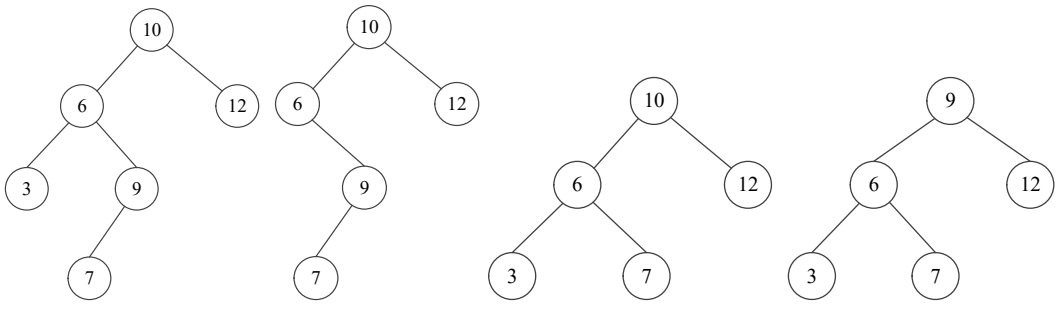


图 7.5 在二叉排序树中删除结点

删除算法如下：

算法 7.8 二叉排序树的删除算法

```
BSTNode *BSTDelete( BSTNode *T, KeyType key )
{
    BSTNode *p=T,*pa=NULL;  // 定义指针变量 p 指向待删除结点，pa 是其父结点
    BSTNode *f,*q;                  // 定义指针变量 q 指向结点 p 的左分支的最大值结点，f 是 q 的父结点
    // 第 1 步：查找 key 所在结点，确定待删除结点及其父结点指向
    while( p )
    {
        if(p->key==key) break;
        else if( p->key<key )     // 进入右分支
        {
            pa = p;
            p = p->right;
        }
        else
        {
            pa = p;
            p = p->left;
        }
    }
    if( !p )   return T;                    // 查找不成功，则直接返回根结点结束
    // 第 2 步：删除结点 p
    //  2.1 第 1 种情况：待删除结点 p 为叶子结点
    if( !p->left && !p->right )
    {
        if( pa )                            // 如果待删除结点非根结点
        {
            if( pa->key<key )           // 如果待删除结点是其父亲的右孩子
                pa->right = NULL;   // 更新父结点的右孩子指向
            else                            // 如果待删除结点是其父亲的左孩子
                pa->left = NULL;    // 更新父结点的做孩子指向
            free(p);                    // 释放待删除结点
            return T;                   // 返回根结点，删除完成
        }
        else                            // 如果待删除结点是根结点
        {
            free(p);                    // 直接释放待删除结点
            return NULL;            // 此时树为空，返回空指针，删除结束
        }
    } // end if  2.1
    // 2.2 第 2 种情况：待删除结点 p 只有右孩子
    else if( !p->left&&p->right )   // 如果待删除结点只有右孩子
    {
        if( pa )                            // 如果待删除结点非根结点
        {
            if( pa->key<key )       // 如果待删除结点是其父亲的右孩子
                pa->right = p->right; // 更新父结点的右孩子指向
            else                            // 如果待删除结点是其父亲的左孩子
                pa->left = p->right;  // 更新父结点的左孩子指向
            free(p);                    // 释放待删除结点
            return T;                   // 返回根结点，删除完成
```

```
    }
    else                          // 如果待删除结点是根结点
    {
        T = p->right;             // 更新根结点指向为其右孩子
        free(p);                  // 释放待删除结点
        return T;                 // 返回根结点，删除完成
    }
} // end else if  2.2
// 2.3 第 3 种情况：待删除结点 p 只有左孩子
else if( p->left && !p->right )
{
    if( pa )                      // 如果待删除结点非根结点
    {
        if( pa->key<key )         // 如果待删除结点是其父亲的右孩子
            pa->right = p->left; //更新父结点的右孩子指向
        else                      // 如果待删除结点是其父亲的左孩子
            pa->left=p->left;     // 更新父结点的左孩子指向
        free(p);                  // 释放待删除结点
        return T;                 // 返回根结点指向，删除完成
    }
    else                          // 如果待删除结点是根结点
    {
        T = p->left;              // 更新根结点为其左孩子
        free(p);                  // 释放待删除结点
        return T;                 // 返回根结点指向，删除完成
    }
} // end else if  2.3
// 2.4 第 4 种情况：待删除结点既有左孩子又有右孩子
else
{
    // 进入待删除结点的左分支，寻找最右端结点,令 q 指向其左分支的最右端结点，f 是 q 的父结点
    f = p;
    q = p->left;
    while( q->right )             // 寻找待删除结点左分支的最右端结点
    {
        f = q;
        q = q->right;
    }
    p->key = q->key;              // 以最右端结点的数值覆盖待删除结点，然后删除该最右端结点
    if( !q->left && !q->right )   // 如果最右端结点是叶子结点
    {
        if( f->key < q->key )
            f->right = NULL;
        else
            f->left = NULL;
        free(q);
    }
    else                          // 如果最右端结点只有左孩子
    {
        if( f->key<q->key )
            f->right = q->left;
        else
            f->left = q->left;
```

```
            free(q);
        }
        return T;                    // 返回根结点，删除结束
    } // end else 2.4
} // BSTDelete
```

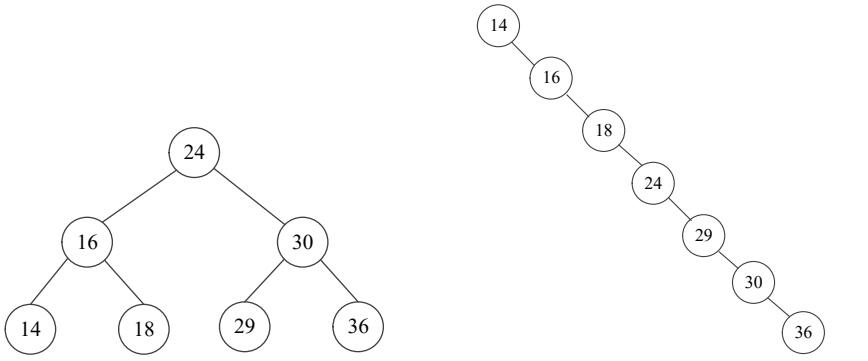
7.4.1.5 二叉排序树的查找性能分析

根据前述算法，在二叉排序树中查找一个包含给定关键字结点的过程相当于走过一条从根结点到该结点的路径，所需要的比较次数与该结点在二叉树中的层次一致。这个查找过程与折半查找很类似。但是，包含n个结点的折半查找的判定树是唯一的，而同样包含n个结点的二叉排序树确并不唯一。如图 7.6 所示，同样是包含 7 个结点，如果输入的关键字序列是（24,16,18,30,14,29,36），那么根据算法 7.7 创建的二叉排序树如图 7.6a 所示，深度为 3；但是如果相同的关键字以（14,16,18,24,29,30,36）的顺序输入，那么创建得到的二叉排序树将是如图 7.6b 所示的形态，深度为 7。如果进行等概率查找，那么在图 7.6a 中的查找成功的平均查找长度为

$$ASL_{(a)} = \frac{1}{7}(1 + 2 \times 2 + 3 \times 4) = 17/7$$

在图 7.6b 中的查找成功的平均查找长度为：

$$ASL_{(b)} = \frac{1}{7}(1 + 2 + 3 + 4 + 5 + 6 + 7) = 28/7 = 4$$



a)关键字序列为（24,16,18,30,14,29,36）　b)关键字序列为（14,16,18,24,29,30,36）

图 7.6 不同形态的二叉排序树

由此可见，包含有n个结点的二叉排序树的平均查找长度与树的形态有关，而二叉排序树的形态与关键字的插入顺序有关。如果关键字有序，那么二叉排序树将是一棵只有左分支或者只有右分支的单支树，树的高度等于n，此时的查找退化为顺序查找，平均查找长度等于$(n+1)/2$，这是最坏的情况；而最好的情况则是二叉排序树的形态与折半查找的判定树一致，树的高度为$\lfloor \log_2 n \rfloor + 1$，此时的平均查找长度近似于$\log_2 n$。

可以证明，在关键字插入顺序随机的情况下，创建得到的二叉排序树的平均查找长度和$\log_2 n$是同数量级的，然而经过进一步的研究发现，这种情况出现的概率不足 50%。因此，需要在创建二叉排序树的过程中进行“平衡化”处理，以得到一棵平衡的二叉排序树。

7.4.2 平衡二叉排序树

7.4.2.1 平衡二叉排序树的定义

平衡二叉排序树又称为 AVL 树（得名于它的发明者 G. M. Adelson-Velskii 和 E. M. Landis），它既可以是一棵空树，也可以是具有如下性质的二叉排序树——其左子树和右子树都是平衡的二

叉排序树，且左、右子树的深度之差的绝对值不超过 1。如果为二叉排序树中的每个结点添加一个平衡因子，并将其定义为该结点左子树的深度与右子树深度的差值，那么 AVL 树中平衡因子的取值只可能是-1、0 和 1。只要二叉树中存在某个结点的平衡因子绝对值大于 1，则该二叉树一定不是平衡的。

我们希望由任何初始的关键字序列创建得到的二叉排序树都是 AVL 树。因为可以证明，包含有n个结点的 AVL 树的深度和$\log n$是同数量级的，因此 AVL 树的平均查找长度也与$\log n$同数量级。

7.4.2.2 平衡二叉排序树的平衡化过程

如何能够得到一棵 AVL 树呢？以关键字序列（14,18,20）为例，如图 7.7 所示：在只包含一个根结点“14”的情况下，二叉排序树是平衡的（图 7.7a）；在插入“18”后也是平衡的，只是根结点的平衡因子变为-1（图 7.7b）；在继续插入“20”后，根结点的平衡因子变为-2，绝对值大于 1，因此树变得不平衡了（图 7.7c），此时需要对树做一些调整，即在不影响二叉排序树基本性质的前提下，将其调整为平衡的二叉排序树（图 7.7d）。由此可见，二叉排序树变得不平衡的原因是，在创建过程中，向平衡的树中插入了一个新结点破坏了树的平衡性。此时应该进行一些**“平衡化”**的处理和调整，使之继续保持平衡的状态。

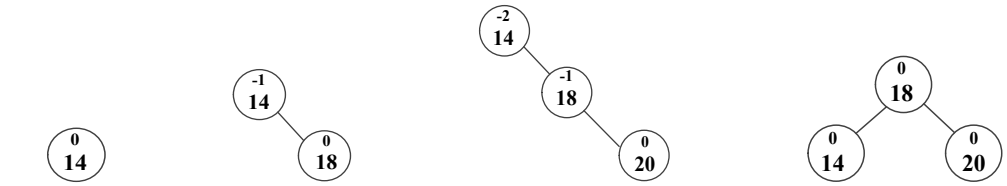


图 7.7 平衡二叉排序树的生成过程

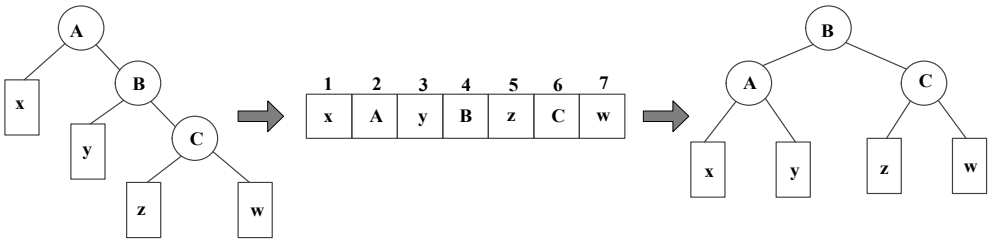
为了进行“平衡化”处理，首先需要找出在插入一个新结点后失去平衡的最小子树的根结点，然后调整这个最小子树中相关结点之间的链接关系，使之成为一个平衡的子树。当失去平衡的最小子树被调整平衡后，整个二叉排序树也就变为平衡的了。失去平衡的最小子树的根结点是指距离新插入的结点最近，且平衡因子的绝对值超过 1 的结点。确定这个最小子树根结点的具体方法是——沿着从新插入结点到二叉排序树根结点的路径，按照从下到上的顺序，依次判断每个结点的平衡因子，在此过程中遇到的第一个平衡因子绝对值大于 1 的结点即为失去平衡的最小子树的根结点；如果一直到根结点的所有结点的平衡因子绝对值皆小于 1，则说明当前的二叉排序树是平衡的，可以继续插入新结点。

在找到了失去平衡的最小子树的根结点后，如何对这棵子树进行“平衡化”的调整呢？假定该最小子树的根结点为 A，在上述确定这个根结点 A 的路径上，A 的孩子假定为 B，B 的孩子假定为 C。A、B 和 C 之间只会存在以下 4 种关系：

（1） RR 型。这是由在 A 的右孩子的右子树中插入了新结点导致的不平衡，如图 7.8a 所示，图中长方形的框表示结点 A、B 和 C 的子树。将图 7.8a 所示的 A、B、C、x、y、z 和 w 按照中序遍历的顺序排列的结果如图 7.8b 所示。在该中序序列中，位于中间（4 号）位置的结点 B 将作为新的子树的根结点；它的左右孩子分别是位于中序序列中的 2 号和 6 号位置上的 A 和 C；A 的左、右孩子分别是在中序序列中的 1 号和 3 号位置的 x 和 y；C 的左、右孩子则分别是在中序序列中的 5 号和 7 号位置的 z 和 w。调整以后的子树如图 7.8c 所示。因为调整前后对应的中序序列相同，所以调整后二叉排序树的性质仍保持不变。可将此调整过程可看成将这棵子树向左旋转一次。

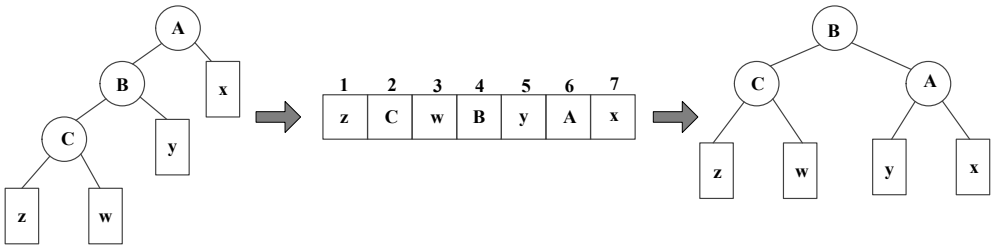
（2） LL 型。这是由在 A 的左孩子的左子树中插入了新结点导致的不平衡，如图 7.9a 所示，图中长方形的框表示结点 A、B 和 C 的子树。与上述 RR 型的调整方案一致，首先将图 7.9a 所示的

A、B、C、x、y、z 和 w 按照中序遍历的顺序排列，结果如图 7.9b 所示。然后根据它们在中序序列中的位置做出相应调整——中间（4 号）位置上的是新的根结点，它的左、右孩子分别是中序序列中的 2 号和 6 号位置上的元素，2 号结点的左、右孩子分别是中序序列中的 1 号和 3 号位置上的元素，而 6 号结点的左、右孩子分别是中序序列中的 5 号和 7 号位置上的元素。调整后的子树如图 7.9c 所示。可将该调整过程看成将这棵子树向右旋转一次。



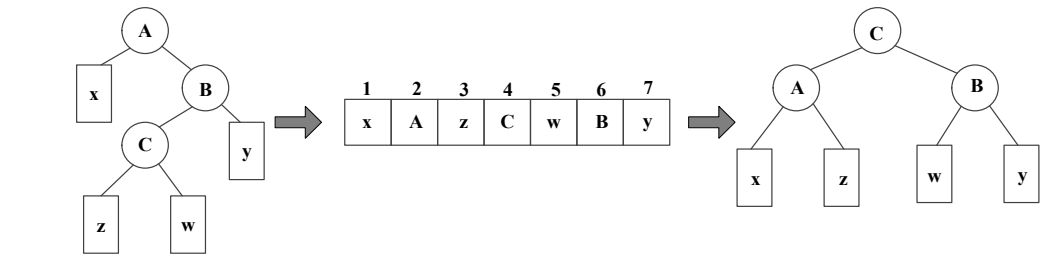
a) 失去平衡的最小子树　　b) 该最小子树的中序序列　　c) 调整后平衡的子树

图 7.8 RR 型调整过程



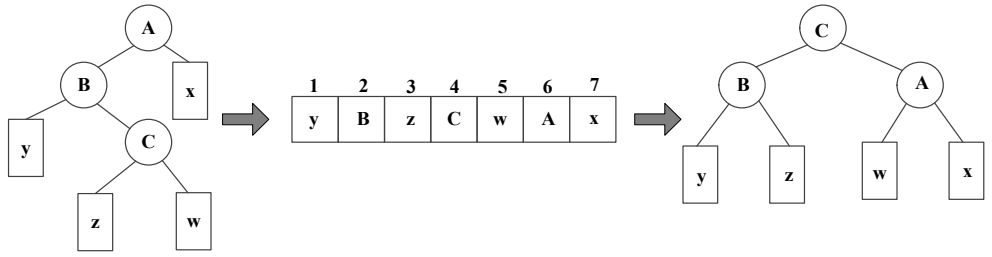
a) 失去平衡的最小子树　　b) 该最小子树的中序序列　　c) 调整后平衡的子树

图 7.9 LL 型调整过程



a) 失去平衡的最小子树　　b) 该最小子树的中序序列　　c) 调整后平衡的子树

图 7.10 RL 型调整过程



a) 失去平衡的最小子树　　b) 该最小子树的中序序列　　c) 调整后平衡的子树

图 7.11 LR 型调整过程

（3） RL 型。这是由在 A 的右孩子的左子树中插入新结点导致的不平衡，如图 7.10a 所示，图中长方形的框表示结点 A、B 和 C 的子树。以同样的方式按照该最小子树的中序序列做出调整，调整结果如图 7.10c 所示。可将该调整过程看成将这棵子树先向右旋转一次再向左旋转一次。

（4） LR 型。这是由在 A 的左孩子的右子树中插入新结点导致的不平衡，如图 7.11a 所示，图中长方形的框表示结点 A、B 和 C 的子树。以同样的方式按照该最小子树的中序序列做出调整，调整后如图 7.11c 所示。可将该调整过程看成将这棵子树先向左旋转一次再向右旋转一次。

在 AVL 树上做查找的过程与一般的二叉排序树相同。那么包含有n个结点的 AVL 树的最大深度是多少呢？我们先来分析深度为h的 AVL 树至少会包含多少个结点。

假定深度为h的 AVL 树最少包含n_h个结点。显然$n_0=0$，$n_1=1$，$n_2=2$，并且$n_h=n_{h-1}+n_{h-2}+1$。这个关系与斐波那契序列极为相似。利用归纳法容易证明：当$h\geq 0$时，$n_h=F_{h+2}-1$，而F_h约等于$\varphi^h/\sqrt{5}$（其中$\varphi^h=\frac{1+\sqrt{5}}{2}$），则$n_h$约等于$\varphi^{h+2}/\sqrt{5}-1$。反之，含有$n$个结点的 AVL 树的最大深度为$\log_\varphi(\sqrt{5}(n+1))-2$。因此，在 AVL 树上进行查找的时间复杂度为$O(\log n)$。

7.4.3 B-树

7.4.3.1 B-树的定义

B-树又称为多路查找树，是一种用于组织和维护外存文件系统的非常有效的数据结构。

在 B-树中，结点分支数目的最大值称为树的阶，通常用m表示。一棵m阶的 B-树既可以是空树，也可以是满足下列条件的m叉树：

（1） 所有的叶子结点都在最底层并为同一层，且不带信息（可以看成是外部结点或查找失败的结点，实际上这些结点并不存在，指向这些结点的指针为空）；

（2） 树中每个结点最多可以有m棵子树（即每个结点最多可以包含$m-1$个关键字）；

（3） 若根结点不是终端结点，则根结点至少有两棵子树（根结点至少包含一个关键字）；

（4） 除根结点外，其它分支结点至少包含$\lceil m/2\rceil$棵子树（每个结点至少包含$\lceil m/2\rceil-1$个关键字）；

（5） 每个分支结点的结构为

n	p_0	k_1	p_1	k_2	p_2	…	k_n	p_n

其中，n表示该结点中包含的关键字的数目，除了根结点外，所有分支结点的关键字个数n必须满足$\lceil m/2\rceil-1\leq n\leq m-1$；$k_i$（$1\leq i\leq n$）为该结点中包含的第$i$个关键字，且满足$k_i<k_{i+1}$；$p_i$（$0\leq i\leq n$）为该结点的第$i$个分支指向，且满足$p_i$所指结点中包含的所有关键字均大于等于$k_i$且小于$k_{i+1}$；

B-树的结点类型定义如下：

```
#define MAX_M  10              // B-树的最大阶数
typedef struct node
{
    int keynum;                // 结点包含的关键字个数
    KeyType key[MAX_M];        // 存放关键字的数组，0 号单元未用
    struct node *parent;       // 父结点指针
    struct node *ptr[MAX_M];   // 存放孩子结点指针的数组
}BTNode, *BTree;               // B-树结点类型定义
```

B-树是所有结点的平衡因子均等于 0 的多路查找树。图 7.12 是一棵 3 阶 B-树，其叶子结点在最底层，根结点只包含一个关键字，其它分支结点均包含 2 个关键字，该树的深度为 4。

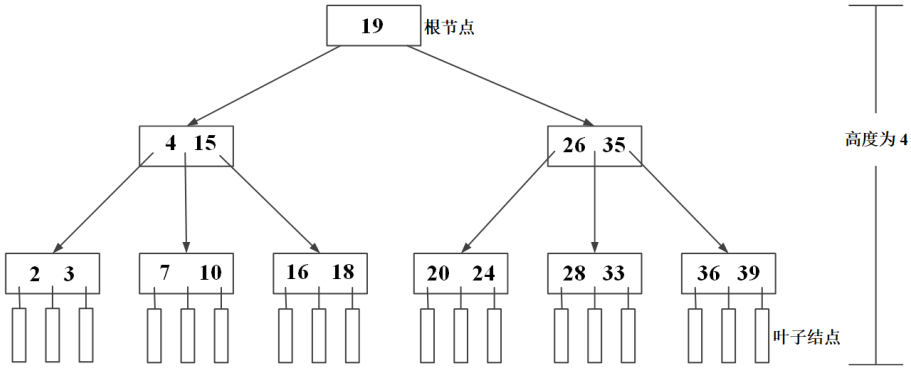


图 7.12　一棵 3 阶的 B-树

7.4.3.2 B-树的查找

在 B-树中查找给定关键字 key 的过程类似于在二叉排序树上进行查找，不同的是在每个结点上确定向下继续查找的路径不一定是二路的，而是多路的。因为一个结点内包含的关键字序列是有序的，因此在一个结点内查找关键字时既可以采用顺序查找也可以采用折半查找。在一棵 B-树上查找关键字为 key 的过程如下：

（1） 首先将 key 与根结点中的k_i进行比较；

（2） 若 key=k_i,则查找成功；

（3） 若 key< k_1，则沿着p_0指向的子树继续查找；

（4） 若k_i <key< k_{i+1}，则沿着p_i指向的子树继续查找；

（5） 若 key> k_n，则沿着p_n指向的子树继续查找；

（6） 若此时p_i指向空，则查找不成功。

算法 7.9 B-树的查找算法

```
typedef struct result
{
    BTNode    *pt;                    // 指向找到的结点
    int       i;                      // 1…m，在结点中的关键字序号
    int       tag;                    // 1: 查找成功，0: 查找失败
} Result;                             // B-树的查找结果类型

Result  SearchBTree( BTree T, KeyType key )
{   // 在 m 阶 B-树 T 上查找关键字 key，返回结果 ( pt, i, tag )。若查找成功，则 tag=1, 指针 pt 所指结点中的
    // 第 i 个关键字等于 key；否则 tag=0, 等于 key 的关键字应插入在指针 pt 所指结点中第 i 和第 i+1 个关键
    // 字之间
    p = T;  q = NULL;  found = FALSE; i = 0;          // 初始化，p 指向待查结点，q 指向 p 的双亲
    while ( p && !found )
    {
        i = Search(p, key);    //在 p->key[1..keynum]中查找，i 使得 p->key[i] <= key < p->key[i+1]
        if ( i>0 && p->key[i]==key) found = TRUE;    // 找到待查关键字
        else  { q=p;  p = p->ptr[i]; }                // 继续下一层结点查找
    }
    if (found)  return (p, i, 1);                     // 查找成功
    else  return ( q, i, 0);                          // 查找不成功，返回 key 的插入位置信息
}  // SearchBTree
```

由此可见，待查关键字所在结点在 B-树上的层次，是决定 B-树查找效率的首要因素。若考虑最坏的情况，即包含n个关键字的m阶 B-树，则可能的最大深度是多少呢？

根据 B-树的定义，其第 1 层至少有一个结点；第 2 层至少有 2 个结点；由于除了根结点之外的每个分支结点至少要有$\lceil m/2 \rceil$棵子树，因此第 3 层至少有$2 \times (\lceil m/2 \rceil)$个结点；以此类推，第$i+1$层至少有$2 \times (\lceil m/2 \rceil)^{i-1}$个结点。而第$i+1$层是叶子结点所在的最底层。若$m$阶 B-树中包含$n$个关键字，则叶子结点为$n+1$，有

$$n+1 \geq 2 \times (\lceil m/2 \rceil)^{i-1} \qquad (7.7)$$

反之

$$i \leq \log_{\lceil m/2 \rceil}\left(\frac{n+1}{2}\right) + 1 \qquad (7.8)$$

也就是说，在包含n个关键字的m阶 B-树上进行查找，最坏情况下需要进行的比较次数不超过$\log_{\lceil m/2 \rceil}\left(\frac{n+1}{2}\right)+1$次。

7.4.3.3 B-树的关键字插入

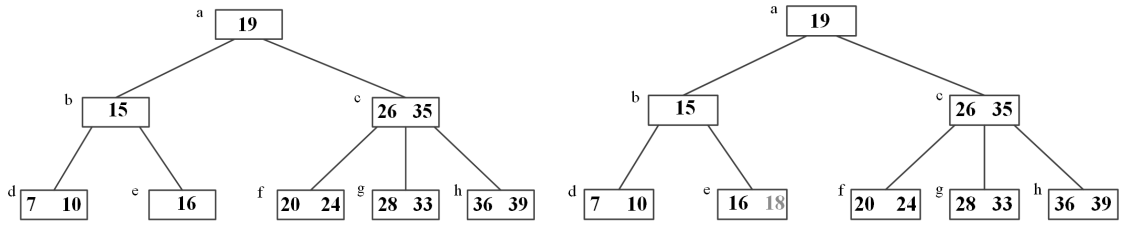
将关键字 key 插入到m阶的 B-树需要分两步来执行：

（1） 查找。利用查找算法找出插入该关键字的最底层的某个分支结点（注意在 B-树中进行 0 插入时，不是在树中新增一个叶子结点，而是在树的最底层的某分支结点中进行插入）。

（2） 插入。判断该分支结点中是否还有空位。

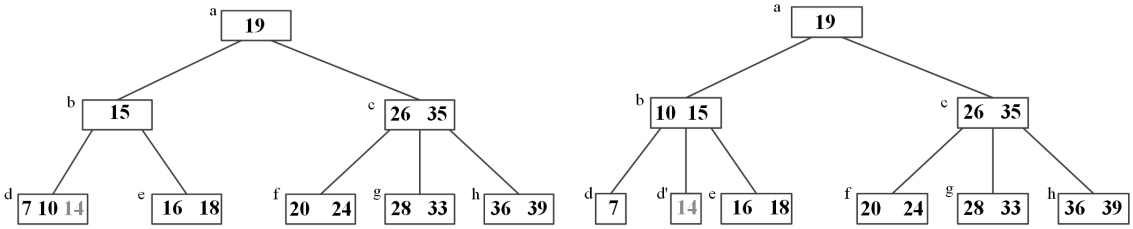
- 不分裂：如果当前分支结点中包含的关键字数量$n < m-1$，则表明当前分支结点中还有多余的空位可以插入关键字 key，此时直接将 key 插入在结点中的合适位置，保证结点中的关键字有序即可。
- 分裂：如果当前分支结点中包含的关键字数量$n = m-1$，说明当前的分支结点中已经没有多余的位置可以插入 key，此时需要将结点做分裂处理。分裂的做法是：将当前结点中的所有关键字与 key 排好序后，从中间位置即$\lceil m/2 \rceil$之处将所有关键字（不包括位于中间位置上的关键字）分成两部分，左边部分的关键字依旧放在原先的分支结点中，右边部分的关键字放在一个新生成的结点中，中间位置上的关键字以及指向新结点的指针一起插入并存储到原分支结点的父结点中。如果父结点的关键字个数也超过$m-1$，则继续分裂，再向上插入，直至进行到根结点为止。这样可能导致 B-树的深度增加一层。

以图 7.13a 所示的 3 阶 B-树为例，要向树中插入关键字“18”。则应由根结点开始查找，最后确定应该插入到结点（16）。该结点当前值包含一个关键字，小于可以包含关键字数量的最大值，因此可以直接将“18”插入该结点，如图 7.13b 所示。



a) 原始的 3 阶 B-树　　b) 插入关键字“18”以后的 B-树

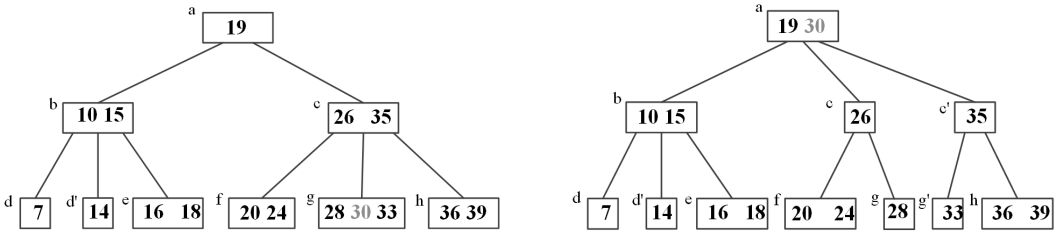
图 7.13　在 3 阶 B-树中插入关键字“18”



a)继续插入关键字“14”　　b)插入关键字“14”以后的 B-树

图 7.14　在 3 阶 B-树中插入关键字“14”

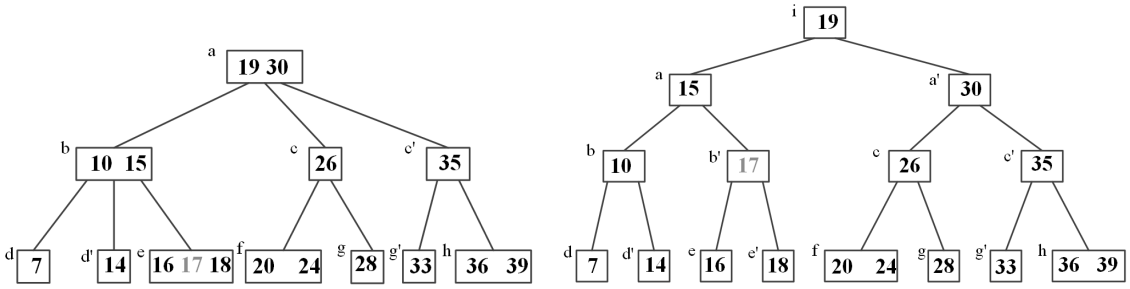
继续插入关键字“14”，通过查找，确定要插入的结点应该是（7,10），如图 7.14a 所示。但是该结点目前已包含两个关键字，即已经没有多余的空位可以插入“14”，因此需要将该结点进行分裂。按照上述的分裂原则，首先将“7,10,14”关键字的左边部分“7”保留在原结点中，然后新生成一个结点存放右半部分关键字“14”，位于中间位置的关键字“10”与指向新结点（14）的指针一起放入原（7,10）结点的父结点（15）中，如图 7.14b 所示。



a)继续插入关键字“30”　　b)插入关键字“30”以后的 B-树

图 7.15　在 3 阶 B-树中插入关键字“30”

继续插入关键字“30”。通过查找，确定要插入的结点应该是在（28, 33），如图 7.15a 所示。但是该结点目前已包含两个关键字，即已经没有多余的空位可以插入“30”，因此需要将该结点进行分裂。按照上述的分裂原则，首先将“28, 30, 33”关键字的左边部分“28”仍然保留在原结点中，然后新生成一个结点存放右半部分关键字“33”，最后将位于中间位置的关键字“30”与指向新结点（33）的指针一起放入原（28, 33）结点的父结点（26, 35）中，但是该结点目前也没有多余的位置可以安放“30”，因此该结点按照同样方式继续分裂，最终得到如图 7.15b 所示的结果。



a)继续插入关键字“17”　　b)插入关键字“17”以后的 B-树

图 7.16　在 3 阶 B-树中插入关键字“17”

假定再继续插入关键字“17”，如图 7.16a 所示，那么结点（16,18）、（10,15）和根结点（19,30）都需要进行分裂，最后的结果是 B-树的深度增加了一层，如图 7.16b 所示。

注意，B-树的生成就是从空树开始，逐个插入关键字而生成的。

7.4.3.4 B-树的关键字删除

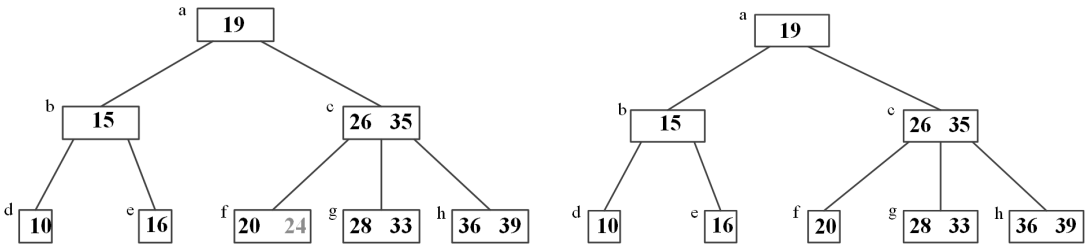
B-树的删除过程与插入过程类似，只是稍微复杂一些。因为要使得删除后的结点所包含的关键字数量不小于$\lceil m/2 \rceil - 1$，所以这将涉及结点的“合并”问题。具体过程为：

（1） 查找：利用查找算法找到要删除关键字 key 所在结点 q，假定 key 等于结点 q 的第i个关键字k_i；

（2） 删除：如果结点 q 是最底层的一个分支结点，那么将k_i从结点 q 中删除；如果结点 q 不在最底层，则用结点 q 中的p_i所指子树中的最小关键字来覆盖k_i，这个最小关键字必然是在最底层的某个分支结点中，因此问题又可以转化为如何在最底层的分支结点中删除一个关键字。在最底层的分支结点 q 中删除关键字k_i，可以分三种情况来讨论。

- 直接删除关键字：如果在结点 q 中删除关键字k_i后，结点包含的关键字数量依然不小于$\lceil m/2 \rceil - 1$，则可以直接删除k_i。
- 向兄弟结点借：如果在结点 q 中删除关键字k_i后，其包含的关键字数量小于$\lceil m/2 \rceil - 1$，但是其左（右）兄弟的关键字数量是大于$\lceil m/2 \rceil - 1$，则可以把其左（右）兄弟中的最大（最小）关键字上移到父结点中，同时把父结点中大于（或小于）上移关键字的关键字下移到结点 q 中。
- 合并：假若结点 q 删除关键字k_i后，其包含的关键字数量小于$\lceil m/2 \rceil - 1$，而且其左（右）兄弟的关键字数量都是等于$\lceil m/2 \rceil - 1$的，即兄弟结点没有多余的关键字可以借出，此时需要将删除关键字k_i后的结点 q 与其左（右）兄弟结点以及父结点中用以分割二者的关键字合并成一个结点。如果此项操作使得父结点的关键字个数也少于$\lceil m/2 \rceil - 1$，则对父结点也做同样的合并处理，可能直到根结点都需要做此处理，从而使得 B-树的高度减少一层。

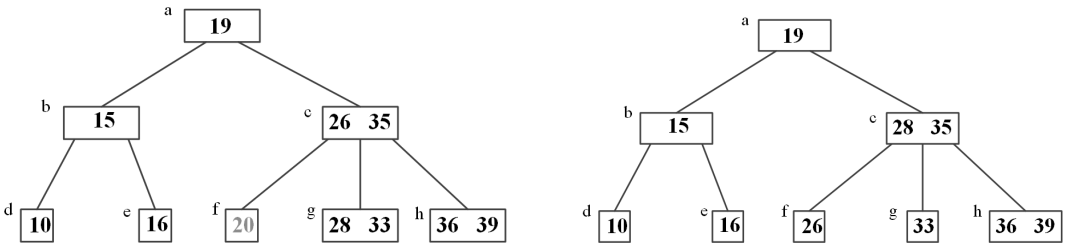
在图 7.17a 所示 3 阶 B-树中删除关键字“24”，它所在的结点（20,24）包含两个关键字，因此可以直接将其进行删除处理，结果如图 7.17b 所示。

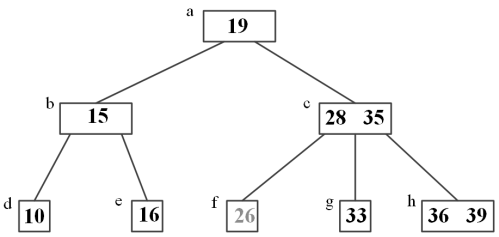


a) 删除关键字“24”　　b) 删除关键字“24”以后的 B-树

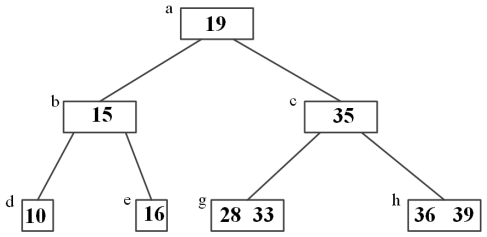
图 7.17　在 3 阶 B-树中删除关键字“24”

继续删除关键字“20”，如图 7.18a 所示。由于包含关键字“20”的结点只有这一个关键字，删除之后该结点包含的关键字数量将少于$\lceil m/2 \rceil - 1$，但是它的右兄弟结点（28,33）有多余的关键字可以借出，因此将其右兄弟中的最小关键字“28”上移至父结点，同时将父结点中小于“28”的关键字“26”移下来，与要删除“20”的结点合并，得到的结果如图 7.18b 所示。





a) 删除关键字“20”



b) 删除关键字“20”以后的 B-树

图 7.18　在 3 阶 B-树中删除关键字“20”

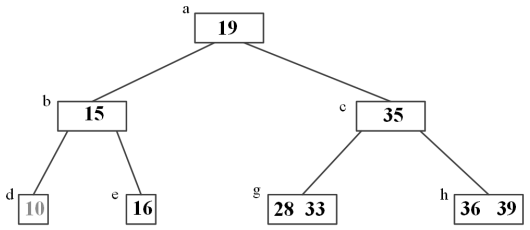
继续删除关键字“26”，如图 7.19a 所示。由于包含关键字“26”的结点只有这一个关键字，删除之后该结点包含的关键字数量将少于$\lceil m/2 \rceil - 1$，但是与它相邻的兄弟结点也没有多余的关键字可以借出，因此将该结点与其右兄弟结点（33）以及父结点中的关键字“28”相合并，得到的结果如图 7.19b 所示。

a) 删除关键字“26”

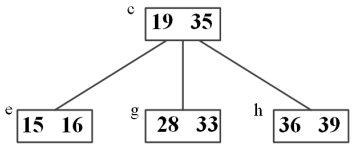
b) 删除关键字“26”以后的 B-树

图 7.19　在 3 阶 B-树中删除关键字“26”

继续删除关键字“10”，如图 7.20a 所示。由于包含关键字“10”的结点只有这一个关键字，删除之后该结点包含的关键字数量将少于$\lceil m/2 \rceil - 1$，但是与它相邻的兄弟结点也没有多余的关键字可以借出，因此只能将该结点与其右兄弟结点（16）以及父结点中的关键字“15”相合并。但是父结点中也只包含了一个关键字“15”，而且其兄弟结点（35）中也没有多余的关键字可以借出，因此需要继续向上，与（15）的父结点再进行合并。由于结点（15）的父结点已经是根结点，而且根结点中也只有一个关键字，因此最终合并的结果如图 7.20b 所示，树的深度减少了一层。



a) 删除关键字“10”



b) 删除关键字“10”以后的 B-树

图 7.20　在 3 阶 B-树中删除关键字“10”

7.4.4 B+树

B+树是 B-树的一种变形。一棵m阶的 B+树要满足以下条件：

（1） 每个分支结点最多可以有m个分支；

（2） 根结点至少有 2 个分支；

（3） 除根结点之外，每个分支结点至少有$\lceil m/2 \rceil$棵子树；

（4） 有n个分支的结点中有n个关键字；

（5） 所有的叶子结点包含全部关键字以及指向相应记录的指针，而且叶子结点按关键字大小顺序链接（可以把每个叶子结点看做是一个索引块，它的指针不再指向另一个索引块，而是直接指向数据文件中的记录）；

（6） 所有的分支结点可以看作是索引部分，其中只包含它的各个分支结点中的最大（或最小）关键字以及指向分支的指针。

图 7.21 是一个 4 阶 B+树，其叶子结点中每个关键字对应的指针指向该记录的实际存储位置。通常 B+树上有两个头指针，一个指向根结点，称为 root；另一个指向关键字最小的叶子结点，称为 sqt。因此，可以对 B+树进行两种查找：一种是从最小关键字开始的顺序查找；另一种是从根结点开始的随机查找。在 B+树上进行随机查找、插入和删除的过程与 B-树基本类似。只是在查找时，若在分支结点上找到待查找关键字，查找过程并不终止，而是继续向下直到叶子结点为止。在 B+树中进行查找时，不管成功与否，每次查找都需要走过一条从根结点到叶结点的路径。

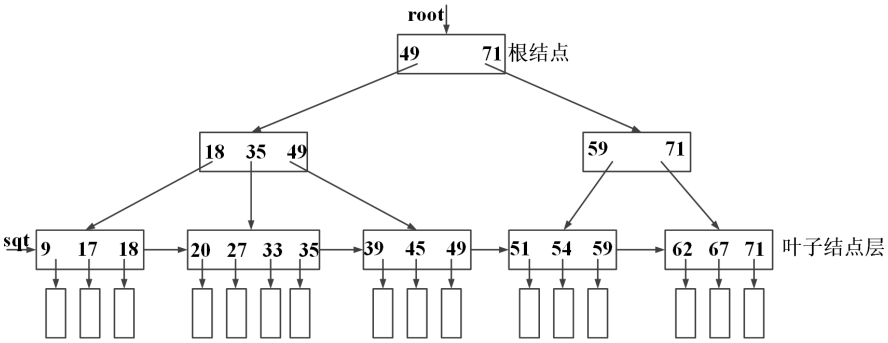


图 7.21　4 阶 B+树

7.4.5 红黑树

平衡二叉排序树虽然保证了查找的性能，但删除结点时的重平衡过程可能需要进行$\log_2 n$次旋转，从而频繁地导致全树整体拓扑结构的大幅度变化。为此，1972 年 Rudolf Bayer 提出了对称二叉 B 树（Symmetric Binary B-tree），后由 Leo J. Guibas 和 Robert Sedgewick 改进，形成了如今的红黑树（Red-black Tree）。红黑树通过对结点指定颜色，并巧妙地动态调整，保证在每次插入和删除结点操作之后的重平衡过程中，全树拓扑结构的更新仅涉及常数个结点。

除了符合二叉排序树的基本要求外，一棵红黑树还需满足以下条件：

（1） 每个结点或是红色或是黑色；

（2） 根结点为黑色；

（3） 外部 NULL 结点均为黑色；

（4） 如果当前结点为红色，其两个孩子结点均为黑色；

（5） 从任一外部结点到根结点的路径上，黑色结点的数目相同。

图 7.22 中给出了一棵红黑树的示例。其中白色背景表示红色结点，深色背景表示黑色结点，N 表示外部 NULL 结点。该棵红黑树满足上述 5 个性质。

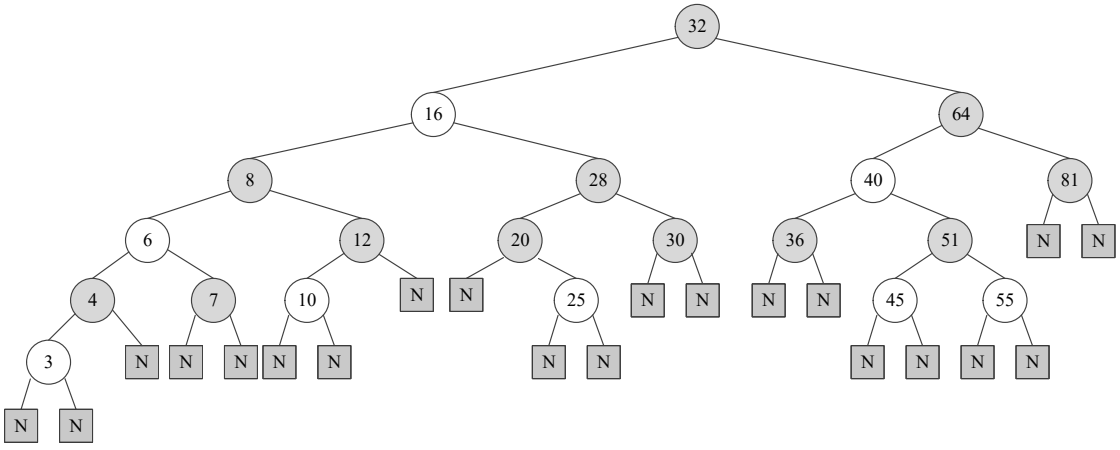


图 7.22　红黑树

根据上述性质 5，可定义红黑树的**黑高度**。将从某个结点 x 出发（不包括该结点）到达一个外结点的任意一条路径上的黑结点个数称为该结点的黑高度。红黑树的黑高度定义为其根结点的黑高度。从根结点到达每个外部 NULL 结点的黑高度相同 。由此可见，红黑树中任一结点左、右子树的高度相差不超过两倍，是一种“适度平衡”。

在红黑树中插入和删除结点时，采用 4 阶 B-树的模式，从而较为方便地实现红黑树的再平衡过程，具体步骤可参阅相关文献。

7.5 哈希表的查找

哈希表的实现常被称为**散列**，散列是一种可以用常数表示的平均时间完成插入、删除和查找的技术。相比于树表，它的优点在于查找算法的时间复杂度进一步减小，接近$O(1)$；但那些需要元素间任何排序信息的操作在哈希表中都得不到有效的支持，比如寻找最大值、寻找最小值或将整个表按顺序输出等。

7.5.1 哈希表的定义

哈希表实际上是一个具有固定大小的连续空间，其中每条记录的存储位置和它的关键字之间有一个确定的对应关系 H，称为**哈希函数**。通过哈希函数的映射得到的关键字在哈希表中的存储位置称为**哈希地址**。理想情况下，这个函数应该计算简单，并且保证可以将任何两个不同的关键字映射到不同的存储单元。但实际上，这是不可能的，因为查找表的长度是有限的。但是关键字可能的取值范围却要大得多。因此，哈希函数往往就是一种压缩映射，将关键字从原本很大的取值空间压缩至较小的查找表内的存储空间，这将导致不同的关键字经过哈希函数的映射后，得到相同哈希地址的现象，称为**冲突**（collision），而产生冲突的关键字也叫作**同义词**。

例如，建立一个江苏省内几个城市的人口统计表，以南京、徐州、镇江、苏州、扬州、常州和无锡为例。假定查找表的表长为 9，其中每条记录都包括城市名、人口总数、70 岁以上老人总数、18 岁以下青少年总数等数据项。设以城市名作为关键字key，哈希函数为$H(key)$ =（key的首字母-‘A’）MOD 9，即城市名的首字母与字母“A”的 ASCII 码的码值相减后与表长做求模运算，以保证得到的哈希地址在表长范围以内。那么南京的首字母是“N”，其哈希函数的取值为 4，也就是说“南京”这条记录在查找表中的哈希地址为“4”。以此类推，徐州、镇江、苏州、扬州和常州这些记录的哈希地址如图 7.23 所示。如果想继续放入“无锡”这条记录，则由于无锡的哈希地址也等于 4，与“南京”产生了冲突，所以无法放入。

	0	1	2	3	4	5	6	7	8
key			CHANGZHOU 常州		NANJING 南京	XUZHOU 徐州	YANGZHOU 扬州	ZHENJIANG 镇江	SUZHOU 苏州

图 7.23 哈希表示例

一般情况下，冲突只可能尽量减少，但无法避免。因此，我们的目标就是寻找一个哈希函数，可以将关键字均匀的分配在哈希表中，剩下的问题就是选择一种合理的策略来解决冲突。

7.5.2 哈希函数的构造方法

哈希函数的设置非常灵活，常用的构造方法有以下几种。

1. **直接定址法**

该方法取关键字或关键字的某个线性函数作为哈希地址，即$H(key) = key$或者$H(key) = a \times key + b$，其中$a$和$b$为常数。这种方法比较简单，适用于当关键字的取值范围与哈希表的表长范围比较接近的情况。

例如，用一个考试成绩统计表统计成绩在 40—100 分的考生人数。如果以成绩作为关键字key，哈希函数就可以取关键字本身，即$H(key) = key$，如表 7.1 所示。此时，关键字的取值范围与哈希表的表长一致，因此不会产生冲突。但在实际应用中，这种情况并不常见。

表 7.1 直接定址法的哈希表

地址	040	041	042	…	060	061	…	099	100
成绩	<=40	41	42	…	60	61	…	99	100
人数	4671	2301	1927	…	4311	3019	…	1003	19

2. 数字分析法

假设关键字是一个r进制的数（如果是十进制，则$r = 10$），可以选取关键字中的若干数位或它们的组合作为哈希地址。这种方法适用于能预先估计出全体关键字的每一位上各种数字出现的频度，并尽量选择那些分布比较均匀的数位的情况。

例如，现有 800 条手机号码的记录，哈希表的表长为 1000。若以手机号码作为关键字key，而手机号码可以看作是一个 11 位的十进制数，则可以取其中的三位数组成哈希地址。那么应该选哪 3 位数呢？原则当然是尽量减少冲突的次数，因此需要对这 800 个手机号码进行分析。我们知道，中国的手机号码，前 3 位是网络识别号，4-7 位代表地区编码，最后的 8-11 位是用户的编码。因此，手机号码的前 3 位只可能是几组固定的数字，可区分性不强；而最后 4 位是随机的，可以从中选取各种数字出现最为平均的 3 位作为哈希地址，也可以将这 4 位数拆开后组成一个 3 位数作为哈希地址。

3. 平方取中法

该方法取关键字平方后的中间几位作为哈希地址。当关键字中的每一个数位上都有某些数字重复出现且频度很高时，数字分析法将难以使用。对关键字求平方可以扩大其差别，同时平方值的中间各位又能受到整个关键字中各个数位的影响，由此得到的哈希地址随机性将更大，可以减少地址的冲突。

例如，以图 7.22 所示的哈希表为例，将哈希函数修改为首先计算关键字首字母的 ASCII 码的平方，然后取平方后的百位数作为哈希地址，如表 7.2 所示。此时根据 7 个关键字得到的 7 个地址将不会产生冲突。

表 7.2 平方取中法

关键字	NANJING 南京	XUZHOU 徐州	ZHENJIANG 镇江	SUZHOU 苏州	YANGHZHOU 扬州	CHANGZHOU 常州	WUXI 无锡
(关键字首字母)^2	6084	7744	8100	6889	7921	4489	7569
哈希地址	0	7	1	8	9	4	5

4. 折叠法

该方法将关键字分割成若干部分，然后取它们的叠加和为哈希地址。此方法适用于关键字的数位特别多的情况。

例如，每一个期刊都有一个连续的国际标准书号（ISBN）（共 13 位），可以将它的后 8 位看作一个 8 位的十进制数。若以该数字作为关键字key建立哈希查找表，表长为 1000，则可采用折叠法构造一个 3 位数的哈希地址。折叠法包括移位叠加和间界叠加两种方法。移位叠加是将分割后的

数直接累加；间界叠加则是将分割后的数间隔着进行逆置后再累加。假设某期刊的 ISSN 后 8 位是10068961，则对其利用折叠法得到的哈希地址如图 7.24 所示。



a)移位叠加　　b)间界叠加

图 7.24　折叠法

5. 除留余数法

该方法取关键字除以某个不大于哈希表表长n的整数p后所得余数为哈希地址，即

$$H(key) = key\ MOD\ p \qquad p \le n \tag{7.9}$$

这是一种最简单、最常用的方法，其中它不仅可以对关键字直接取模，也可以在数字分析、平方取中或折叠运算之后取模。

在选择这种方法时，p的取值很重要，因为若p取值不当，则很容易出现冲突。一般情况下，我们选择一个不大于n的素数作为p。例如，假设有一组关键字为（12, 28, 18, 24, 36, 22，34，46），如果取p =8，那么这些关键字对应的哈希地址将是（4, 4, 2, 0, 4, 6，2，6），产生了较多冲突。

6. 随机数法

该方法选择一个随机函数，取关键字的随机函数值作为哈希地址，即$H(key) = random(key)$，其中$random$为随机函数。通常，当关键字的长度不等时，可以采取这种方法构造哈希函数。

在实际应用中，采用何种构造哈希函数的方法取决于关键字集合的情况（包括关键字的范围和形态）以及表长范围，总的原则是使关键字均匀分布在哈希表中，并且将产生冲突的可能性尽可能降低。

7.5.3 处理冲突的方式

既然好的哈希函数也只能减少冲突的次数而无法避免冲突，那么必须采用有效的方式来处理冲突。

7.5.3.1 链地址法

解决冲突的第一种方法是链地址法，其做法是将产生冲突的所有关键字保留到一个链表中。此时，哈希表中的每个存储单元存放的并不是记录本身，而是产生冲突的所有关键字构成的单链表的表头。

例如，已知哈希表的表长为 11，一组关键字为（0,1,4,9,16,25,36,49,64,81），根据哈希函数$H(key) = key\ MOD\ 11$并采用链地址法解决冲突最终创建的哈希表如图 7.25 所示。在进行查找时，应根据哈希函数来确定究竟该遍历哪个链表。

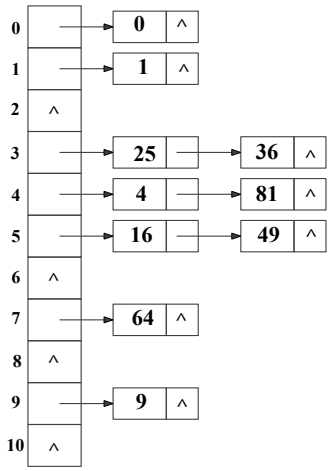


图 7.25　采用链地址法解决冲突的哈希表

7.5.3.2 开放定址法

开放定址法则在顺序存储的哈希表中为产生冲突的地址寻找下一个哈希地址。在处理冲突的过程中，可能会得到一个地址序列H_i（$H_i \in [0, n-1], i = 1,2,\ldots,n-1$,其中$n$表示哈希表的表长），即在处理冲突时寻找的另一个哈希地址H_1仍然是冲突的，只得继续求下一个地址H_2，直到H_j不发生冲突为止。在开放定址法中，以发生冲突的哈希地址为自变量，通过某种函数得到一个新的空闲哈希地址的方法有很多种，下面介绍几种常用的方法。假定关键字的个数为m，哈希表的表长为n，且$m < n$。

1. 线性探测法

该方法以发生冲突的哈希地址开始，依次探测其下一个地址（当到达查找表的表尾时，下一个探测地址应该是首地址“0”），直到找到一个空闲的哈希地址为止。线性探测法的数学递推公式为

$$H_i(key) = (H(key) + i)\ MOD\ n \quad (1 \le i \le n-1) \tag{7.10}$$

线性探测法容易产生堆积问题。因为如果连续出现若干同义词，则这些同义词将会在哈希表中占据连续的存储地址。例如，假设第一个同义词的地址在d，则如果后面连续出现k个同义词，那么这些同义词将会占用哈希表中的地址$d+1, d+2,\ldots, d+k$。之后，任何出现在地址$d+1$到地址$d+k$上的哈希映射，即使没有同义词也会由于之前同义词的堆积产生冲突。为解决这一问题，可以将递推公式改进为

$$H_i(key) = (H(key) + c \times i)\ MOD\ n \qquad (c \ge 1, 1 \le i \le n-1) \tag{7.11}$$

2. 平方探测法

平方探测法的数学递推公式为

$$H_i(key) = (H(key) \pm i^2)\ MOD\ n \quad (1 \le i \le \tfrac{n-1}{2}) \tag{7.12}$$

平方探测法可以避免出现堆积问题。但是要注意，在使用该方法时，应确保哈希表的表长n必须是一个形如$4j+3$的素数（例如 7、11、19、23 等）。因为在处理冲突的过程中，地址序列H_i必须具有“完备性”，即如果当前产生冲突的哈希地址为d，那么H_i应该确保可以覆盖哈希表中除了d以外的所有地址。

例如，若表长$n = 11$，当前的哈希地址$d = 4$，若该地址有冲突，那么在用平方探测法寻找下一个空闲地址时，H_i的取值情况将如表 7.3 所示（做减法时，若出现负值，可以将该负值与表长n

或n的倍数相加，得到正值后再做求模运算）。可以发现，H_i可以覆盖表中除了地址“4”以外的所有剩余地址，因此也确保可以为当前发生冲突的关键字找到一个空闲地址。

表 7.3 表长为 11 时的平方探测法

i	1		2		3		4		5	
地址增量	+1	-1	+4	-4	+9	-9	+16	-16	+25	-25
H_i	5	3	8	0	2	6	9	10	7	1

如果表长为$n = 13$，那么由于 13 虽然也是素数，但不满足$4j + 3$的形式，所以若此时当前的哈希地址$d = 4$，且该地址有冲突，则在用平方探测法来寻找下一个空闲地址时，H_i的取值情况如表 7.4 所示。可以看到，H_i的取值并不完备，哈希表中的地址“2”“6”“9”“10”“11”“12”均未被探测到，因此无法确保可以为当前发生冲突的关键字找到一个空闲地址。

表 7.4 表长为 13 的平方探测法

i	1		2		3		4		5		6	
地址增量	+1	-1	+4	-4	+9	-9	+16	-16	+25	-25	+36	-36
H_i	5	3	8	0	0	8	7	1	3	5	1	7

3. 随机探测法

随机探测又叫作双哈希函数探测，其数学递推公式为

$$H_i(key) = (H(key) + i \times H_2(key))\ MOD\ n \quad (1 \le i \le n-1) \qquad (7.13)$$

其中$H_2(key)$是另设的一个哈希函数，为确保H_i必须具有“完备性”，$H_2(key)$的取值应该与表长n互为素数。若表长n本身就为素数，则$H_2(key)$可以取 1 至$n-1$之间的任意数；若表长n为 2 的幂次，则$H_2(key)$ 应是 1 至 $n-1$ 之间的任意奇数。

7.5.4 哈希表的查找

在哈希表中进行查找的过程与构造哈希表的过程类似。以开放定址法为例，对于给定的待查关键字key，首先根据设定的哈希函数求得相应的哈希地址，若该地址为空，则查找不成功；否则将待查关键字key与该地址中存储的关键字做比较。若相等，则查找成功；否则根据冲突解决机制寻找下一个地址H_i，直至某个H_i为空，或者其中的关键字等于待查关键字key为止。

开放定址法的哈希表存储结构定义为：

```
typedef struct
{
    ElemType *elem;  // 记录存储的基地址，动态分配数组
    int count;       // 查找表中包含的记录个数
    int tablesize;   // 查找表的容量
}HashTable;          // 哈希表类型定义
```

在基于开放定址法构造的哈希表H中查找关键字key，如果存在，则用d表示其存储位置，查找成功，否则查找不成功。

算法 7.10 哈希表的查找算法

```
Status SearchHash( HashTable *H, KeyType key, int &d )
```

```
{
    p = Hash(key);                                          // 根据设定的哈希函数获取哈希地址
    while( H->elem[p]!=NULL && H->elem[p].key!=key )        // 如果地址不空且其中的关键字不等于 key
        p = collision(p);                                   // 根据设定的冲突解决机制获取下一个哈希地址
    if( H->elem[p]!=NULL && H->elem[p].key==key )           // 如果当前地址中的关键字等于 key
    {
        d = p;                                              // 用 d 存储 key 所在的数组位置
        return SUCCESS;                                     // 查找成功
    }
    else
        return UNSUCCESS;                                   // 查找不成功
} // SearchHash
```

如果某个记录e并不存在，则可以将其插入到哈希表中。插入的过程与查找类似，只不过是在查找不成功时，会将e插入到当前为空的哈希地址处。

算法 7.11 哈希表的插入算法

```
Status InsetHash( HashTable *H, ElemType e, int &d )
{
    p = Hash(key);                                          // 根据设定的哈希函数获取哈希地址
    while( H->elem[p]!=NULL && H->elem[p].key!=e.key )      // 如果地址不空且其中的关键字不等于 e.key
        p=collision(p);                                     // 根据设定的冲突解决机制获取下一个哈希地址
    if( H->elem[p].key==e.key )                             // 如果当前地址中的关键字等于 e.key
        return UNSUCCESS;                                   // 插入不成功
    else                                                    // 否则表示当前地址为空
    {
        H->elem[p] = e;                                     // 将记录 e 插入当前位置
        (H->count)++;                                       // 哈希表中包含的记录个数加 1
        d = p;                                              // 用指针 d 保存插入位置
        return SUCCESS;                                     // 插入成功
    }
} // InsertHash
```

值得注意的是，如果在基于开放定址法构造的哈希表中删除一个记录，则需要在该位置上填入一个特殊标记，而不能直接将该位置清空，以免找不到在其后填入的同义词记录。如果是基于链地址法构造的哈希表，则可以直接将对应的结点从链表中删除。

7.5.5 性能分析

从前述的哈希表查找过程可见：

（1） 虽然哈希表的记录与其存储地址之间存在直接的哈希映像关系，但由于冲突的存在，哈希查找并不能在常数时间内完成，依然需要将待查关键字与查找表中的若干关键字进行比较，因此仍然需要利用平均查找长度ASL来衡量哈希表的查找效率；

（2） 查找过程中需要的比较次数与三个因素有关：哈希函数、冲突解决机制以哈希表的装填因子。哈希表的装填因子定义为

$$\alpha = \frac{\text{哈希表中填入的记录个数}}{\text{哈希表长}} \qquad (7.14)$$

装填因子表示了哈希表的填满程度。显然，α越小，发生冲突的可能性就越小；反之α越大，说明表中已经填入很多记录，如果再填入，发生冲突的可能性当然也就越高。

哈希函数的好坏的最直观的影响就是发生冲突的频率。当装填因子以及哈希函数相同时，不同的冲突解决机制将构造出不同的哈希表，相应的平均查找长度也不会不同。

例如，一组关键字（21, 19, 54, 62, 7, 32, 29, 44, 15），哈希表的表长为 11，哈希函数为 $H(key) = key\ MOD\ 11$，如果用链地址法解决冲突，构造的哈希表如图 7.26 所示。

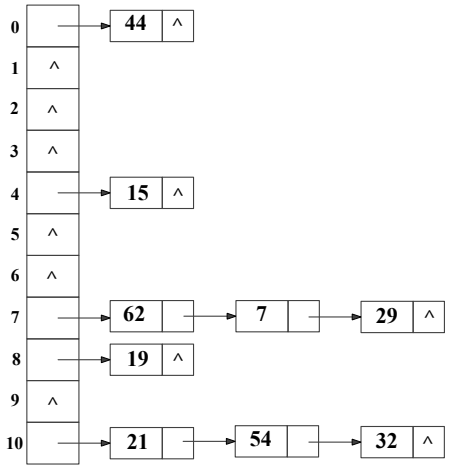


图 7.26 链地址法构造的哈希表

查找每个关键字所需的比较次数如表 7.5 所示。

表 7.5 在基于链地址法构造的哈希表中查找各关键字所需比较次数

关键字	21	19	54	62	7	32	29	44	15
比较次数	1	1	2	1	2	3	3	1	1

由此可以计算该表的平均查找长度为：

$$ASL_{\text{链地址法}} = \frac{1}{9}(1+1+2+1+2+3+3+1+1) = 1.67$$

若采用开放定址法，分别利用线性探测、平方探测以及双哈希函数探测（可以设定$H_2(key) = (3 \times key)MOD10 + 1$）来解决冲突，则得到的哈希表以及查找各关键字所需的比较次数分别如表 7.6、7.7 和 7.8 所示。

表 7.6 基于线性探测法构造的哈希表以及查找各关键字所需比较次数

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	**54**	**32**	**29**	**44**	15	-	-	62	19	**7**	21
比较次数	2	3	7	4	1			1	1	3	1

表 7.7 基于平方探测法构造的哈希表以及查找各关键字所需比较次数

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	**54**	**44**	-	**29**	15	-	**7**	62	19	**32**	21
比较次数	2	2		5	1		3	1	1	3	1

表 7.8 基于双哈希探测法构造的哈希表以及查找各关键字所需比较次数

地址	0	1	2	3	4	5	6	7	8	9	10

关键字	44	-	**54**	**29**	**15**	**32**	62	19	**7**	21
比较次数	1		2	2	3	2	1	1	2	1

可以分别计算这三种不同冲突解决机制下的平均查找长度:

$$ASL_{\text{线性探测法}} = \frac{1}{9}(2+3+7+4+1+1+1+3+1) = 2.56$$

$$ASL_{\text{平方探测法}} = \frac{1}{9}(2+2+5+1+3+1+1+3+1) = 2.11$$

$$ASL_{\text{双哈希函数探测法}} = \frac{1}{9}(1+2+2+3+2+1+1+2+1) = 1.67$$

可以看出，线性探测法容易产生堆积，使得后续的哈希地址不同的记录又会产生新的冲突，从而导致平均查找长度较大。而链地址法、平方探测法以及双哈希函数探测法则可以较好地解决这个问题。但是线性探测的优点在于计算简单。

在哈希函数以及冲突解决机制相同的情况下，平均查找长度则取决于装填因子α。表 7.9 列出了几种不同冲突解决机制下的平均查找长度。从中可以看出，哈希表的平均查找长度不是表长n的函数，而是装填因子α的函数，因此在设计哈希表时，可以选择一个适当的装填因子α，使得平均查找长度限定在某个范围内。

表 7.9 不同冲突解决机制下的哈希表平均查找长度

冲突解决机制	平均查找长度	
	查找成功	查找不成功
链地址法	$1+\frac{\alpha}{2}$	$\alpha + e^{-\alpha} \approx \alpha$
线性探测法	$\frac{1}{2}(1+\frac{1}{1-\alpha})$	$\frac{1}{2}(1+\frac{1}{(1-\alpha)^2})$
平方探测法	$-\frac{1}{\alpha}\ln(1-\alpha)$	$\frac{1}{1-\alpha}$

哈希表的使用非常广泛。例如编译器会使用哈希表来跟踪源程序中声明的变量，这些变量一般都不长，而且也没必要将它们做排序处理，因此用哈希表可以快速地完成这些变量的查找。此外，网络中的域名服务器（DNS）也利用哈希函数来快速的查找由用户输入的域名所对应的 IP 地址。

7.6 小结

本章主要讨论了顺序表、树表和哈希表的查找。

对于顺序表的查找，如果关键字key是无序的，则需要对整个顺序表进行遍历查找，时间复杂度为$O(n)$。如果关键字 key 是有序的，则可以采用折半查找方式，将时间复杂度降低到$O(\log n)$。当然，还可以用索引查找的方式，将数据进行分段，以分级进行查找，避免遍历整个数据表。

相应地，二叉排序树、平衡二叉树则利用了二叉树的性质，将关键字key的查找由整个数据表的遍历变成了从根结点开始，沿着二叉树的左、右子树逐层向下的搜索过程，查找的时间复杂度则取决于树的深度。如果二叉树排序树建立时已退化成单链表形式，树的深度达到n，则查找时间复杂度就退化到$O(n)$。为避免这一问题，平衡二叉树通过二叉排序树建立过程中的平衡化处理，保证了查找算法的时间复杂度为$O(\log n)$。进一步，B-树和 B+树中每个结点可存储更多的数据元

素，以降低树的深度，进而降低查找的时间复杂度。同时，树形结构的查找表也便于结点的插入和删除。

为了更进一步降低查找的时间复杂度，通过建立关键字key值和查找表存储地址的映射关系构造特定的哈希表，则可将查找时间复杂度降到$O(1)$。当然，选择的哈希函数和解决冲突的方法对基于哈希表的查找时间复杂度影响较大。

7.7 练习

1. 在常用的描述二叉排序树的存储结构中，关键字值最大的结点（）。
   A. 左指针一定为空　　B. 右指针一定为空
   C. 左右指针均为空　　D. 左右指针均不为空

2. 对线性表进行折半查找时，要求线性表必须（）。
   A. 以顺序方式存储　　B. 以链接方式存储
   C. 以顺序方式存储且结点按关键字有序排序
   D. 以链接方式存储且结点按关键字有序排序

3. 折半查找和二叉排序树的时间性能（）。
   A. 一定相同　　B. 有时相同
   C. 完全不同　　D. 以上都不对

4. 采用折半查找方式查找一个长度为 n 的表时，其平均查找长度为（）。
   A. O(n)　　B. $O(logn)$
   C. $O(nlogn)$　　D. $O(n^2)$

5. 平衡二叉树，左右子树的高度差最大为（）。
   A. 0　　B. 1
   C. 2　　D. 以上都不正确

6. m 阶 B 树是一棵（）
   A. m 叉查找树　　B. m 叉高度平衡查找树
   C. m-1 叉高度平衡查找树　　D. m+1 叉高度平衡查找树

7. 下面关于 m 阶 B 树的说法中正确的是（）。
（1）每个结点至少有两棵非空子树。
（2）每个结点至多有 m-1 个关键字。
（3）所有叶结点在同一层次上。
（4）当插入一个关键字引起 B 树结点分裂后，树就一定长高一层。
   A.（1）（2）（3）　　B.（2）（3）
   C.（2）（3）（4）　　D.（3）

8. 下列关于 m 阶 B 树的说法错误的是（）。
   A. 根结点至多有 m 棵子树　　B. 所有叶结点在同一层次上
   C. 非终端结点至少有 m/2 （m 为偶） 或 m/2+1（m 为奇）棵子树
   D. 根结点中的数据是有序的

9. 具有 n 个关键字的 m 阶 B 树有（）个终端结点。

A. $n+1$　　B. $n-1$

C. $n \times m$　　D. $logn$

10. 在一棵 m 阶 B 树的结点中插入关键字，若插入前关键字数目为（），则插入后该结点会发生分裂。

A. m　　B. m-1

C. m+1　　D. m-2

11. 如果一棵 m 阶 B 树中删除关键字导致结点需要与其右兄弟或左兄弟合并，那么被删除关键字所在结点在删除关键字之前关键字数目为（）。

A. m-1　　B. $\lceil m/2 \rceil - 1$

C. $\lceil m/2 \rceil$　　D. $\lceil m/2 \rceil + 1$

12. 哈希方法存储的基本思想是根据（）来决定元素的存储地址。

A. 元素的序号　　B. 元素的个数

C. 关键字值　　D. 非码属性

13. 设一个哈希表中有 n 个元素，用哈希方法进行查找的平均查找长度是（）。

A. $O(1)$　　B. $O(n)$

C. $O(n^2)$　　D. $O(logn)$

14. 在构建哈希表时，常会产生冲突，此时的冲突是指（）。

A. 两个元素有相同的序号　　B. 两个元素的关键字值不同，而非关键字值相同

C. 不同关键字值对应到相同的存储地址　D. 装填因子过大，数据元素过多

15. 关于哈希函数，下列说法不正确的是（）

A. 函数应是简单的，能在较短时间内计算出结果

B. 定义域应包括全部关键字值，值域必须在表长范围内

C. 计算出来的地址应该分布在整个地址空间中

D. 装填因子必须限制在 0.8 以下

16. 哈希表的平均查找长度跟（）有关。【多选】

A. 处理冲突的方法　　B. 表的长度

C. 装填因子　　D. 以上都不是

17. 已知一个有序递增的数组 A[1,⋯,4n]，序列中关键字均不相同。按如下方法查找关键字为 k 的记录：先在序号为 4、8、12,⋯4n 的记录中进行查找，可能查找成功，也可能由此确定一个继续查找的范围。请设计满足上述过程的查找算法，并分析算法的时间复杂度。

18. 设计算法，以从大到小输出二叉排序树中不小于 k 的所有关键字。

19. 设计算法判定一棵二叉树是否为二叉排序树。

20. 设计算法判断一棵二叉树是否为平衡二叉树。

21. 给定关键字序列（23, 45, 29, 76, 54, 61, 19, 33, 89, 25, 30），创建一个平衡的二叉排序树和一个 3 阶 B-树。

22. 现有关键字序列（6, 10, 17, 20, 23, 53, 42, 54, 57），已知哈希表长为 11，哈希函数为 $H(key) = (2 \times key)\ MOD\ 11$，利用平方探测法解决冲突。请构造该哈希表，并分别计算在等概率情况下，查找成功与不成功的平均查找长度。

23. 实现二叉查找树中结点的插入和删除。

24. Hash 表应用

[问题描述]

设计散列表实现 VIP 客户发掘。对身份证号进行 Hash, 通过对乘客某时间段内的乘机频率、里程数统计，发掘 VIP 客户。

[基本要求]

（1）设每个记录有下列数据项：身份证号码（虚构，位数和编码规则与真实一致即可）、姓名、航班号、航班日期、里程。

（2）从文件输入各记录，以身份证号码为关键字建立散列表。

（3）分别采用开放定址（自行选择和设计定址方案）和链地址两种方案解决冲突；显示发生冲突的次数、每次中解决冲突进行重定位的次数。

（4）记录条数至少在 100 条以上。

（5）从记录中实现乘客乘机频率、里程数统计，从而发掘 VIP 客户。

25. 平衡二叉树操作的演示

[问题描述]

利用平衡二叉树实现一个动态查找表。

[基本要求]

（1）从文件读取数据

（2）实现动态查找表的三种基本功能：查找、插入和删除。

（3）以可验证的方式输出结果

7.8 延伸阅读

二分查找最早由 John W. Mauchly 于 1946 年在摩尔学院讲座中提出，1960 年由 Derrick H. Lehmer 整理成型并发表[1]。插值查找由 Yehoshua Perl、Alon Itai 等于 1978 年提出[2]。斐波那契查找则由 Mordecai Avriel 和 Douglass J. Wilde 于 1966 年提出[3]。

二叉查找树是由 P. F. Windley、Andrew D. Booth、Andrew Colin、Thomas N. Hibbard 等研究人员在 1960 年提出的。AVL 树由 G. M. Adelson-Velsky 和 E. M. Landis 于 1962 年提出[4]。

散列表是由 IBM 计算机科学家 Hans P. Luhn 于 1953 年撰写的一份 IBM 内部备忘录中提出[5]。

参考文献：

[1] LEHMER D H. Teaching combinatorial tricks to a computer[C]. Proceedings of Symposia in Applied Mathematics. American Mathematical Society. 1960: 179-193.
[2] PERL Y, ITAI A, AVNI H. Interpolation search - a log log N search[J]. Communications of the ACM, 1978, 21(7): 550-553.
[3] AVRIEL M, WILDE D J. Optimality proof for the symmetric Fibonacci search technique[J]. The Fibonacci Quarterly, 1966, 4(3): 265-269.
[4] ADELSON-VELSKY G M, LANDIS E M. An algorithm for the organization of information[J]. Proceedings of the USSR Academy of Sciences, 1962, 146: 263-266 (in Russian). English Translation in Soviet Mathematics Doklady, 1962, 3: 1259-1263.
[5] STEVENS H. Hans Peter Luhn and the birth of the hashing algorithm[J]. IEEE Spectrum, 2018, 55(2): 44-49.